


UNIVERSITÀ
POLITECNICA
DELLE MARCHE

Corso di Laurea Magistrale in Ingegneria Elettronica
Esame di Digital Adaptive circuits and learning systems

---

# Implementazione di un sistema di Small-Footprint Keyword Spotting basato su TinyML per piattaforme embedded STM32

Candidato:
**Luca Ciaramicoli**
**Giulia Marchetti**

Anno Accademico 2025-2026

Università Politecnica delle Marche
Corso di Laurea Magistrale in Ingegneria Elettronica
Esame di Digital Adaptive circuits and learning systems
Via Brecce Bianche – 60131 Ancona (AN), Italy

# Indice

**1 Introduzione** **1**
1.1 Obiettivo del progetto . . . . . 1
1.2 Stato dell'arte . . . . . 2
1.2.1 Approccio discriminativo basato sull'apprendimento supervisionato per il KWS . . . . . 4
1.2.2 Tecniche di apprendimento . . . . . 5
1.2.3 Compressione del modello . . . . . 6
1.2.4 Sviluppo di SF-KWS su schede . . . . . 6

**2 Materiali e Metodi** **9**
2.1 Rete Neurale . . . . . 9
2.2 Dataset: Google Speech Commands Dataset (GSCD) . . . . . 13
2.3 Coefficienti MFCC per il riconoscimento vocale . . . . . 13

**3 Svolgimento del progetto** **17**
3.1 Fase 1: Download in locale del Google Speech Commands . . . . . 17
3.2 Fase 2: Suddivisione del dataset in Training, Validation e Test . . . 17
3.3 Fase 3: Estrazione delle feature MFCC dai file audio . . . . . 18
3.4 Fase 4: Creazione della rete DS-CNN . . . . . 20
3.5 Fase 5: Addestramento . . . . . 21
3.5.1 Configurazione dell'ambiente di sviluppo . . . . . 21
3.5.2 Parametri di addestramento . . . . . 22
3.5.3 Risultati dell'addestramento . . . . . 23
3.5.4 Prove con architetture alternative . . . . . 24
3.5.5 Matrici di confusione . . . . . 27
3.6 Fase 6: Conversione del modello in tflite . . . . . 28
3.7 Fase 7: Preparazione dei dati di validazione in formato NPY . . . . 30
3.8 Fase 8: Analisi del modello con STM32Cube AI Studio . . . . . 30
3.9 Fase 9: Validazione on desktop e on target . . . . . 31
3.10 Fase 10: Confronto tra rete a 2 blocchi e rete a 4 blocchi . . . . . 34

**Conclusioni** **37**

# Elenco delle figure


1.1 *Il kit STEVAL-BCNKT01V1 BlueCoin usato come piattaforma target* 1
1.2 *Panoramica del sistema: KWS eseguito sul dispositivo edge vs. ASR basato su servizi cloud [1].* . . . . . 2
1.3 *Confronto tra la dimensione del modello e l'accuratezza per i modelli software (Keras) e le relative varianti TFLite, evidenziando le differenze in termini di efficienza e prestazioni del modello [1].* . . . 7
1.4 *Confronto del tempo di inferenza dei modelli software (Keras) vs. modelli TFLite, mostrando la riduzione del tempo di esecuzione per i modelli TFLite attraverso diverse architetture [1].* . . . . . 8

2.1 *Keyword spotting pipeline [2].* . . . . . 9
2.2 *Architettura della rete utilizzata.* . . . . . 12
2.3 *Tre rappresentazioni dello stesso segnale audio, un suono di motosega. In alto: segnale audio nel tempo; al centro: spettrogramma Mel; in basso: MFCC.* . . . . . 15

3.1 *Parte dello script di estrazione del dataset Google Speech Commands* 18
3.2 *Visualizzazione di alcune feature MFCC estratte per ogni classe target* 19
3.3 *Architettura della rete DS-CNN utilizzata* . . . . . 20
3.4 *Accuratezza e loss su training e validation set della rete originale a quattro blocchi* . . . . . 23
3.5 *Accuratezza e loss della rete a quattro blocchi con dropout 0.5* . . . . 24
3.6 *Accuratezza e loss della rete a due blocchi con dropout 0.2* . . . . . 25
3.7 *Accuratezza e loss della rete a due blocchi con dropout 0.7* . . . . . 25
3.8 *Accuratezza e loss della rete a dodici blocchi con dropout 0.2* . . . . . 25
3.9 *Matrice di confusione sul validation set della rete originale a quattro blocchi* . . . . . 27
3.10 *Matrice di confusione sul validation set della rete a due blocchi* . . . 28
3.11 *Riepilogo di STM32Cube AI Studio per la rete a 2 blocchi: occupazione di flash e RAM e metriche di validazione* . . . . . 31
3.12 *Riepilogo di STM32Cube AI Studio per la rete a 4 blocchi: occupazione di flash e RAM e metriche di validazione* . . . . . 32
3.13 *Hardware usato per la validazione on target* . . . . . 33
3.14 *Tempo di inferenza per layer e occupazione della RAM della rete a 2 blocchi, misurati on target* . . . . . 34

3.15 *Tempo di inferenza per layer e occupazione della RAM della rete a 4 blocchi, misurati on target* . . . . . 35

# Elenco delle tabelle

2.1 Riepilogo delle migliori reti neurali per ciascuna classe di vincoli hardware (S, M, L) [2]. . . . . 10

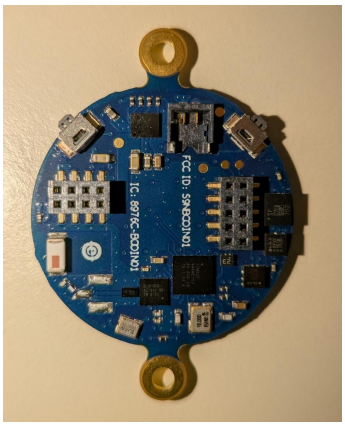
3.1 *Versioni dei componenti necessarie per l'addestramento su GPU locale in ambiente Windows* . . . . . 21
3.2 *Accuratezza e loss su training e validation set in alcune epoche significative. In grassetto il modello salvato.* . . . . . 23
3.3 *Confronto tra le architetture testate* . . . . . 26
3.4 *Risultati dell'analisi con STM32Cube AI Studio delle due reti quantizzate* 30
3.5 *Risultati della validazione on target sul test set (4.888 campioni)* . . 34

# Capitolo 1

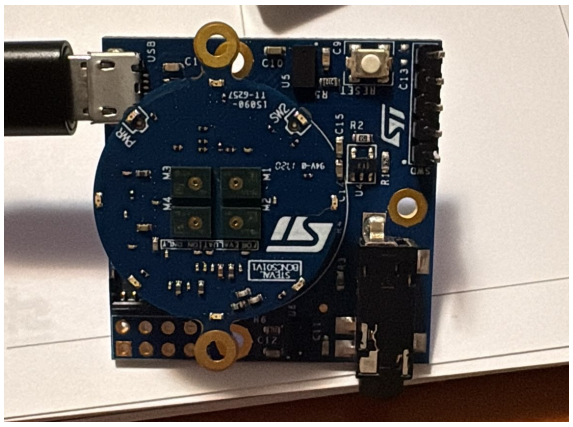
# Introduzione

## 1.1 Obiettivo del progetto

Il progetto ha l'obiettivo di riconoscere 10 parole chiave (*keyword spotting*), più le classi silenzio e parole sconosciute per un totale di 12 classi, a partire da un segnale audio acquisito tramite microfono. Il riconoscimento è affidato a una DS-CNN eseguita sul kit STEVAL-BCNKT01V1 BlueCoin basato su microcontrollore STM32F446, mostrato in Figura 1.1. La piattaforma target dispone di risorse computazionali estremamente limitate, pari a 512 KB di memoria Flash e 128 KB di RAM, rendendo necessario un processo di ottimizzazione particolarmente accurato rispetto alla fase di sviluppo su un normale computer. Proprio per questi limiti di memoria e di calcolo, il progetto si colloca nell'ambito del *Small-Footprint Keyword Spotting* (SF-KWS).



(a) *Scheda BlueCoin: lato componenti, con il microcontrollore STM32F446 e il modulo Bluetooth BlueNRG*



(b) *BlueCoin montata sulla CoinStation; al centro i quattro microfoni MEMS M1–M4*

Figura 1.1: *Il kit STEVAL-BCNKT01V1 BlueCoin usato come piattaforma target*

## 1.2 Stato dell'arte

Il Keyword Spotting (KWS) [1] consiste nel riconoscimento di una o più parole predefinite all'interno di un flusso audio continuo e costituisce una delle principali tecnologie alla base dei sistemi di attivazione degli assistenti vocali. In questo contesto operativo, illustrato nella Figura 1.2, un utente può avviare il sistema pronunciando una parola di attivazione (*wake-up word*), come "Ok Google" nel caso dei dispositivi Google Home; successivamente, il sistema può stabilire una connessione con i servizi cloud per l'elaborazione delle richieste.

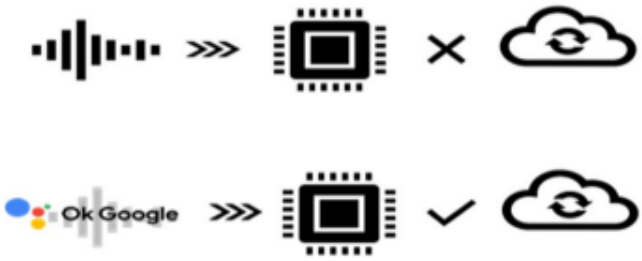



Figura 1.2: *Panoramica del sistema: KWS eseguito sul dispositivo edge vs. ASR basato su servizi cloud [1].*

A differenza dell'*Automatic Speech Recognition* (ASR), il cui obiettivo è la trascrizione continua del parlato e che, a causa della maggiore complessità computazionale, viene frequentemente eseguito su infrastrutture remote, il KWS deve essere in grado di operare direttamente sul dispositivo edge. Tale caratteristica risponde alle esigenze di bassa latenza, ridotto consumo energetico e tutela della privacy, evitando la trasmissione continua del segnale audio verso il cloud. La necessità di operare in presenza di risorse computazionali e di memoria estremamente limitate ha portato allo sviluppo dello *Small-Footprint Keyword Spotting* (SF-KWS), un ambito di ricerca focalizzato sulla progettazione di modelli sufficientemente compatti da poter essere eseguiti su dispositivi quali microcontrollori, smartwatch e sistemi per l'Internet of Things (IoT), spesso caratterizzati da una memoria flash inferiore a 2 MB. Pertanto, per implementare un modello SF-KWS su dispositivi edge, risulta fondamentale il ricorso a tecniche e framework di *Tiny Machine Learning* (TinyML). Storicamente, i primi approcci al KWS si basavano su sistemi costituiti da *Hidden Markov Models* (HMM) associati alla decodifica di Viterbi, una procedura computazionalmente onerosa e scarsamente adatta a dispositivi caratterizzati da risorse limitate. L'introduzione delle *Deep Neural Networks* (DNN) e, più in generale, del *Deep Learning* (DL) ha consentito di sviluppare modelli discriminativi in grado di stimare direttamente le probabilità a posteriori delle classi, ad esempio distinguendo tra parole chiave, rumore e parlato di fondo, riducendo la necessità di procedure di ricerca sequenziale complesse. Rispetto agli approcci basati su HMM, le tecniche di DL hanno evidenziato diversi vantaggi:

- l'utilizzo di una tecnica di elaborazione a posteriori più semplice rispetto ad algoritmi di ricerca sequenziale complessi;
- maggiore adattabilità ai vincoli delle risorse computazionali e di memoria;
- prestazioni superiori rispetto agli approcci basati su HMM, in particolare nelle applicazioni caratterizzate da risorse hardware limitate.

Tra le architetture di DL, le *Convolutional Neural Networks* (CNN) hanno assunto un ruolo rilevante grazie alla loro struttura compatta e alla capacità di eseguire in modo efficiente le operazioni convoluzionali.

Un ulteriore aspetto di interesse riguarda l'impiego di tecniche di TinyML per l'ottimizzazione dei modelli destinati ai dispositivi edge caratterizzati da risorse computazionali e di memoria limitate. Framework quali *TensorFlow Lite Micro*, *STM32Cube.AI* ed *Edge Impulse* consentono di implementare ed eseguire l'inferenza in tempo reale su microcontrollori dotati di una memoria inferiore a 1 MB. L'ottimizzazione dei modelli può essere ottenuta attraverso diverse tecniche, tra cui il **quantization-aware training (QAT)**, il **pruning** e la **knowledge distillation**, che rientrano nelle tecniche di compressione dei modelli. Tali approcci permettono di ridurre l'occupazione di memoria e il costo computazionale, favorendo l'implementazione dei sistemi KWS su piattaforme embedded. In particolare, l'implementazione del KWS mediante tecniche di TinyML su dispositivi edge presenta diversi vantaggi:

- **Bassi requisiti di potenza**: consente di operare efficientemente su dispositivi edge a basso consumo, rendendoli adatti per applicazioni alimentate a batteria;
- **Bassa latenza**: il KWS può rispondere rapidamente alle parole chiave, rispetto ai sistemi tradizionali di riconoscimento vocale, senza dover attendere l'elaborazione dell'intera espressione vocale;
- **Dimensioni compatte del modello**: i modelli KWS destinati al TinyML sono ottimizzati per ridurre l'occupazione di memoria, consentendone la distribuzione su dispositivi con capacità di archiviazione limitata;
- **Elaborazione in tempo reale**: consente il rilevamento tempestivo delle parole chiave, rendendo possibile una risposta immediata del sistema;
- **Capacità di edge computing**: l'elaborazione può essere eseguita direttamente sul dispositivo, senza la necessità di una connessione costante a un server centralizzato. L'elaborazione locale contribuisce, inoltre, alla tutela della privacy, evitando la trasmissione continua dei dati audio;
- **Parole chiave personalizzabili**: alcune soluzioni permettono anche di personalizzare le parole chiave o di aggiornare il modello direttamente sulla scheda [1]. In questo progetto il modello viene addestrato su PC e poi soltanto eseguito sulla scheda.

### 1.2.1 Approccio discriminativo basato sull'apprendimento supervisionato per il KWS

L'approccio discriminativo al KWS si basa tipicamente sull'apprendimento supervisionato, nel quale il modello viene addestrato utilizzando un dataset di segmenti audio associati alle rispettive etichette. Il sistema riceve in ingresso il segnale vocale e, attraverso una fase di estrazione delle caratteristiche, ne ricava una rappresentazione compatta nel dominio tempo-frequenza. Tali caratteristiche vengono successivamente elaborate da un modello di classificazione, generalmente basato su DNN, che produce una distribuzione di probabilità sulle diverse classi, corrispondenti alle parole chiave e, in genere, a categorie quali rumore di fondo e parlato non contenente parole target. Il processo di KWS può essere implementato secondo due modalità principali: non-streaming e streaming. Nella modalità non-streaming, ogni segmento audio contenente una singola parola viene classificato indipendentemente, selezionando la classe associata alla probabilità a posteriori più elevata. Nella modalità streaming, invece, il sistema analizza continuamente il flusso audio, elaborando finestre temporali successive e producendo una sequenza di probabilità. Poiché tali finestre possono risultare rumorose e fortemente correlate nel tempo, vengono generalmente sottoposte a procedure di smoothing. Il rilevamento della parola chiave può quindi essere effettuato mediante una soglia di sensibilità oppure selezionando la classe con probabilità maggiore all'interno di una finestra temporale. Per evitare il rilevamento multiplo della stessa parola, vengono inoltre adottati meccanismi di blocco temporaneo successivi all'attivazione.

Una fase fondamentale è rappresentata dall'estrazione delle feature del segnale audio. Tra le rappresentazioni maggiormente utilizzate in questo ambito figurano gli spettrogrammi *log-Mel* e i *Mel-Frequency Cepstral Coefficients* (MFCC), ottenuti attraverso l'analisi spettrale del segnale mediante la Trasformata di Fourier Veloce (FFT) e banchi di filtri definiti sulla scala Mel. Queste rappresentazioni permettono di descrivere in forma compatta le caratteristiche spettrali e temporali del segnale audio. Prima di essere fornite al modello, vengono generalmente sottoposte a procedure di normalizzazione, al fine di favorire la stabilità del processo di addestramento e migliorare la capacità di generalizzazione. Nelle applicazioni destinate a dispositivi edge, è fondamentale contenere la complessità computazionale associata all'estrazione delle feature. Per questo motivo, è possibile ricorrere a rappresentazioni compatte e a tecniche di riduzione della precisione numerica, che consentono di limitare l'impiego delle risorse disponibili. In alternativa agli approcci tradizionali, sono stati proposti metodi basati sull'apprendimento automatico della rappresentazione, tra cui gli *autoencoder*. Questi modelli sono capaci di apprendere automaticamente caratteristiche informative direttamente dai dati audio, riducendo la necessità di definire manualmente le rappresentazioni del segnale.

Una volta estratte le feature, queste vengono fornite a un classificatore che determina la probabilità di appartenenza alle diverse classi. Le architetture basate sul

*Deep Learning* hanno progressivamente sostituito gli approcci tradizionali, grazie alla maggiore accuratezza e robustezza, in particolare in presenza di rumore. I primi sistemi KWS basati sul *Deep Learning* utilizzavano principalmente DNN completamente connesse, successivamente affiancate da CNN e *Recurrent Neural Network* (RNN). Le CNN sfruttano le convoluzioni per estrarre pattern locali nel dominio tempo-frequenza, mentre le RNN, tra cui LSTM e GRU, sono adatte a modellare le dipendenze temporali del segnale. Per l'impiego del KWS su dispositivi edge e sistemi embedded, caratterizzati da risorse computazionali e di memoria limitate, particolare attenzione è rivolta alla progettazione di architetture compatte ed efficienti. Tra queste, le *Depthwise Separable Convolutional Neural Network* (DS-CNN) riducono il costo computazionale delle convoluzioni tradizionali mediante la separazione delle operazioni *depthwise* e *pointwise*. Un'altra soluzione è rappresentata dalle *Temporal Convolutional Network* (TCN), che utilizzano convoluzioni dilatate per ampliare il campo ricettivo temporale mantenendo contenuto il numero di operazioni. Infine, tecniche di decomposizione tensoriale consentono di approssimare filtri di grandi dimensioni mediante strutture più compatte, contribuendo alla riduzione della memoria occupata e del costo computazionale.

Nel complesso, l'approccio discriminativo supervisionato al KWS combina l'estrazione delle caratteristiche, la modellazione acustica e la gestione delle probabilità a posteriori, con l'obiettivo di identificare in modo accurato e tempestivo le parole chiave all'interno di un flusso audio continuo. La progettazione di modelli compatti ed efficienti rappresenta un elemento fondamentale per consentirne l'implementazione su dispositivi embedded caratterizzati da basse risorse.

### 1.2.2 Tecniche di apprendimento

L'efficienza del KWS può essere migliorata intervenendo sulle modalità di addestramento. Tra le tecniche maggiormente utilizzate rientrano la *Knowledge Distillation* (KD), l'apprendimento auto-supervisionato (*Self-Supervised Learning*, SSL) e l'augmentazione dei dati, che consiste nella manipolazione dei dati in ingresso per distorcere e modificare leggermente le feature, rendendo i modelli più resistenti all'overfitting e permettendo di ampliare artificialmente il dataset. La *Knowledge Distillation* permette di trasferire la conoscenza di un modello insegnante di grandi dimensioni a un modello studente più compatto. In questo modo è possibile ottenere modelli con un numero significativamente inferiore di parametri, mantenendo prestazioni comparabili a quelle del modello originale. L'apprendimento auto-supervisionato, invece, consente di sfruttare grandi quantità di dati audio non etichettati per apprendere rappresentazioni utili al KWS, riducendo la dipendenza da dataset annotati. Queste tecniche risultano particolarmente interessanti poiché intervengono principalmente durante la fase di addestramento e non introducono necessariamente un aumento del costo computazionale durante l'inferenza. In alcuni approcci, inoltre, KD, SSL e

tecniche di compressione vengono combinate per ottenere modelli simultaneamente accurati, compatti e robusti al rumore.

### 1.2.3 Compressione del modello

La compressione dei modelli costituisce un elemento fondamentale per la distribuzione del KWS su dispositivi edge. Tra le principali tecniche impiegate in questo ambito figurano il *pruning* e la quantizzazione, che consentono di ridurre le dimensioni del modello e il costo computazionale associato alla sua esecuzione. Il *pruning* consiste nell'eliminazione dei pesi meno rilevanti di una rete neurale, con l'obiettivo di ottenere una rappresentazione più sparsa e ridurre il numero di parametri effettivamente utilizzati. Tale processo viene generalmente applicato in modo iterativo, alternando la rimozione dei pesi a successive fasi di riaddestramento, al fine di recuperare eventuali perdite di accuratezza introdotte dalla semplificazione della rete. La quantizzazione, invece, consiste nella riduzione della precisione numerica utilizzata per rappresentare i pesi e le attivazioni del modello. Il passaggio da rappresentazioni floating-point a interi a 8 bit, ad esempio, permette di ridurre significativamente l'occupazione di memoria e il costo computazionale, favorendo l'esecuzione del modello su piattaforme embedded. Tra le tecniche di quantizzazione si distingue la *Post-Training Quantization* (PTQ), che può essere applicata al modello al termine della fase di addestramento senza richiedere un ulteriore processo di training. Tuttavia, questa procedura può introdurre una perdita di accuratezza. Per limitare tale effetto, è possibile ricorrere alla *Quantization-Aware Training* (QAT), durante la quale gli effetti della quantizzazione vengono simulati già nel corso dell'addestramento, in particolare durante il *forward pass*, consentendo al modello di adattarsi alla ridotta precisione numerica. Il *pruning* e la quantizzazione, infine, possono essere combinati con ulteriori tecniche di ottimizzazione, quali la decomposizione a basso rango e la *Knowledge Distillation*. L'impiego di questi approcci consente di ottenere modelli particolarmente compatti, cercando di preservarne il più possibile le prestazioni in termini di accuratezza e affidabilità.

### 1.2.4 Sviluppo di SF-KWS su schede

La distribuzione dello SF-KWS su dispositivi embedded si inserisce nel paradigma del TinyML, il cui obiettivo è consentire l'esecuzione di modelli di *machine learning* (ML) su microcontrollori e sistemi caratterizzati da un basso consumo. Diverse piattaforme hardware supportano lo sviluppo di applicazioni TinyML, tra cui Arduino Nano 33 BLE Sense, SparkFun Edge, Adafruit EdgeBadge, Himax WE-I Plus EVB e Raspberry Pi Pico, oltre ai microcontrollori STM32, come lo STM32F446 usato in questo progetto. L'esecuzione dei modelli su tali dispositivi è resa possibile da framework specifici, quali TensorFlow Lite for Microcontrollers (TFLM), CMSIS-NN, Embedded Learning Library (ELL), Edge Impulse e STM32Cube AI Studio, quest'ultimo usato qui per portare il modello sulla scheda. Questi strumenti consentono

di convertire, ottimizzare e distribuire modelli di ML su piattaforme con risorse computazionali e di memoria limitate. Nel caso dello SF-KWS, tale processo permette quindi di trasferire un modello di riconoscimento di keyword vocali da un ambiente di addestramento a una piattaforma embedded, cercando di mantenere un compromesso tra accuratezza, occupazione di memoria, latenza e consumo energetico. Nell'ambito dello sviluppo di sistemi SF-KWS destinati a dispositivi edge per applicazioni IoT, sono stati valutati diversi framework e modelli TinyML, utilizzando il Google Speech Commands Dataset (GSCD) per la classificazione di dieci classi. Le architetture considerate comprendono reti DNN, CNN, DS-CNN e MicroNet, suddivise in modelli di piccole dimensioni (S) e di grandi dimensioni (L). Sono stati inoltre analizzati modelli basati su MFCC e convoluzioni unidimensionali (1D) implementati mediante Edge Impulse.

I modelli sviluppati in Keras sono stati successivamente convertiti e quantizzati in formato Int8 tramite TensorFlow Lite (TFLite). Tale procedura ha consentito di ottenere una riduzione delle dimensioni del modello fino al 69%, associata ad una perdita minima di accuratezza e ad una significativa diminuzione del tempo di inferenza. I risultati evidenziano che i modelli TensorFlow Lite CNN-S e DS-CNN-S risultano particolarmente adatti alla distribuzione su dispositivi edge con una capacità di memoria limitata. Tra i modelli convertiti in TFLite, MicroNet-S ha raggiunto la migliore accuratezza, pari al 95,3%, dimostrando un buon compromesso tra accuratezza e requisiti di memoria. La Figura 1.3 illustra la relazione tra accuratezza e il numero di parametri dei modelli analizzati, includendo sia le implementazioni software basate su Keras sia le rispettive versioni convertite in TFLite.

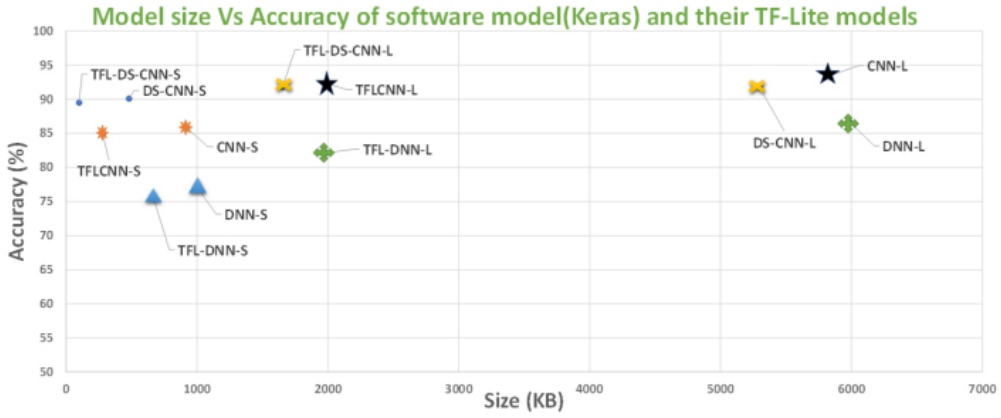



Figura 1.3: *Confronto tra la dimensione del modello e l'accuratezza per i modelli software (Keras) e le relative varianti TFLite, evidenziando le differenze in termini di efficienza e prestazioni del modello [1].*

Inoltre, la Figura 1.4 evidenzia come TensorFlow Lite consenta non solo di ridurre significativamente la dimensione del modello, ma anche di migliorare la velocità di inferenza attraverso la quantizzazione dei pesi, mantenendo al contempo le prestazioni di classificazione comparabili a quelle del modello originale.

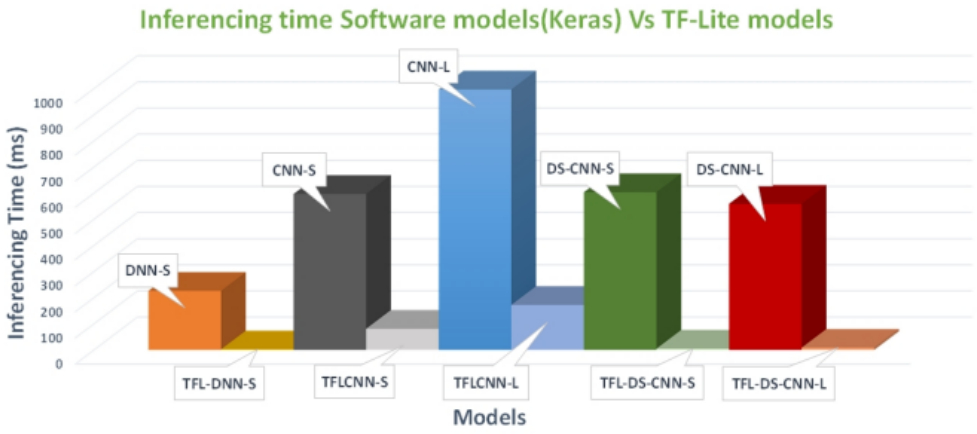



Figura 1.4: *Confronto del tempo di inferenza dei modelli software (Keras) vs. modelli TFLite, mostrando la riduzione del tempo di esecuzione per i modelli TFLite attraverso diverse architetture [1].*

Le applicazioni del KWS si sono progressivamente estese oltre i tradizionali ambiti di utilizzo degli assistenti intelligenti. In ambito sanitario, questi sistemi consentono l'interazione a mani libere, ossia di interagire con un dispositivo senza doverlo toccare o utilizzare manualmente, ma semplicemente pronunciando una parola o un comando specifico, utile nell'assistenza agli anziani e nello sviluppo di tecnologie assistive. Nel settore automobilistico, supportano i sistemi di controllo vocale, contribuendo a migliorare la sicurezza durante la guida, mentre negli scenari industriali basati su IoT facilitano le interazioni tra uomo e macchina. Le principali prospettive di sviluppo riguardano il miglioramento dell'efficienza computazionale e delle prestazioni dei sistemi in condizioni operative reali. In questo contesto, l'obiettivo è rendere lo SF-KWS sempre più adatto all'esecuzione in tempo reale direttamente su dispositivi edge a basso consumo energetico, migliorandone l'efficienza, la tutela della privacy e la capacità di adattamento. Queste caratteristiche assumono particolare rilevanza in applicazioni quali le smart home, i dispositivi indossabili e i sistemi embedded.

# Capitolo 2

# Materiali e Metodi

## 2.1 Rete Neurale

Un tipico sistema KWS [2] è costituito da un estrattore di feature e da un classificatore basato su una rete neurale, come mostrato nella Figura 2.1.

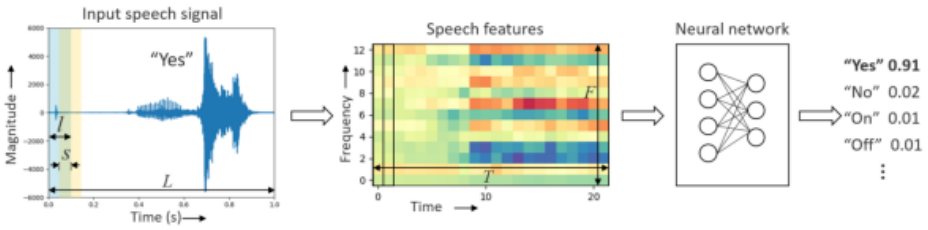



Figura 2.1: *Keyword spotting pipeline [2].*

Innanzitutto, il segnale vocale in ingresso, di lunghezza **L**, viene suddiviso in frame sovrapposti di lunghezza $l$, con uno stride $s$, ottenendo così un numero totale di frame **T** pari a:

$$T = \frac{L - l}{s} + 1 \tag{2.1}$$

Con $L = 1000$ ms, $l = 40$ ms e $s = 20$ ms la formula dà 49 frame, gli stessi di Hello Edge[2]. La libreria SpeechBrain, però, aggiunge un padding di mezza finestra all'inizio e alla fine del segnale, quindi si ottengono 51 frame.

Da ciascun frame vengono estratte **F** feature vocali, ottenendo complessivamente $T \times F$ feature per l'intero segnale in ingresso. Tra le feature acustiche più comunemente utilizzate nei sistemi di riconoscimento vocale basati sul deep learning figurano le energie del banco di filtri log-Mel (*Log-Mel Filter Bank Energies*, LFBE) e i coefficienti *MFCC*. Queste feature, progettate manualmente, derivano dalle tecniche tradizionali di elaborazione del parlato. L'estrazione delle feature tramite LFBE o MFCC consente di trasformare il segnale vocale dal dominio del tempo a una rappresentazione tempo-frequenza, una matrice $T \times F$ (nel caso degli MFCC le feature sono coefficienti cepstrali), permettendo così di ridurre la dimensionalità del segnale in ingresso.

La matrice di feature vocali ottenuta viene quindi fornita al modulo classificatore, che restituisce le probabilità associate alle diverse classi di output. In applicazioni reali, in cui le parole chiave devono essere individuate all'interno di un flusso audio continuo, viene inoltre impiegato un modulo di post-elaborazione che calcola la media delle probabilità di output di ciascuna classe su un determinato intervallo temporale. Questo procedimento contribuisce a migliorare l'affidabilità complessiva della previsione.

Tra le architetture analizzate, la rete *DS-CNN* ha mostrato le migliori prestazioni negli scenari caratterizzati da vincoli hardware considerati in [2]. Tali scenari riflettono le caratteristiche di microcontrollori reali, contraddistinti da risorse di memoria limitate, con una capacità di SRAM compresa tra 8 KB e 320 KB e una memoria Flash disponibile tra 128 KB e 1 MB. In base alle risorse hardware disponibili, gli scenari sono suddivisi in tre categorie: small (S), medium (M) e large (L), come mostrato nella Tabella 2.1. La scheda usata in questo progetto ha 512 KB di Flash e 128 KB di RAM: per la Flash è vicina allo scenario L (500 KB), mentre la RAM è un vincolo più stretto.

| **NN model** | **S (80KB, 6MOps)** | | | **M (200KB, 20MOps)** | | | **L (500KB, 80MOps)** | | |
|---|---|---|---|---|---|---|---|---|---|
| | Acc. | Mem. | Ops | Acc. | Mem. | Ops | Acc. | Mem. | Ops |
| DNN | 84.6% | 80.0KB | 158.8K | 86.4% | 199.4KB | 397.0K | 86.7% | 496.6KB | 990.2K |
| CNN | 91.6% | 79.0KB | 5.0M | 92.2% | 199.4KB | 17.3M | 92.7% | 497.8KB | 25.3M |
| Basic LSTM | 92.0% | 63.3KB | 5.9M | 93.0% | 196.5KB | 18.9M | 93.4% | 494.5KB | 47.9M |
| LSTM | 92.9% | 79.5KB | 3.9M | 93.9% | 198.6KB | 19.2M | 94.8% | 498.8KB | 48.4M |
| GRU | 93.5% | 78.8KB | 3.8M | 94.2% | 200.0KB | 19.2M | 94.7% | 499.7KB | 48.4M |
| CRNN | 94.0% | 79.7KB | 3.0M | 94.4% | 199.8KB | 7.6M | 95.0% | 499.5KB | 19.3M |
| DS-CNN | 94.4% | 38.6KB | 5.4M | 94.9% | 189.2KB | 19.8M | 95.4% | 497.6KB | 56.9M |

Tabella 2.1: Riepilogo delle migliori reti neurali per ciascuna classe di vincoli hardware (S, M, L) [2].

L'idea alla base della DS-CNN, mutuata dall'architettura MobileNet e conseguentemente da AlexNet, sviluppata per la visione artificiale, consiste nello scomporre la tradizionale convoluzione tridimensionale in due operazioni separate e più leggere: una convoluzione depthwise, che applica un singolo filtro 2D a ciascun canale della mappa di feature in ingresso in modo indipendente, e una successiva convoluzione pointwise (con kernel 1×1), che combina linearmente gli output dei singoli canali lungo la dimensione di profondità. Questa scomposizione riduce sensibilmente sia il numero di parametri sia il numero di operazioni rispetto a una convoluzione standard, consentendo di costruire reti più profonde e più ampie pur rimanendo entro i vincoli di memoria e calcolo tipici dei microcontrollori.

Nell'implementazione adottata in [2], la rete è composta da uno strato di convoluzione iniziale seguito da una sequenza di strati DS-Conv (ciascuno costituito dalla coppia depthwise + pointwise, con batch normalization e attivazione ReLU), per poi terminare con un average pooling globale e uno strato completamente connesso finale. L'average pooling globale riduce ogni mappa di feature a un singolo valore,

così lo strato finale ha meno parametri.

Nel caso di questo progetto, l'architettura della rete DS-CNN da noi utilizzata è rappresentata nella Figura 2.2. Le 12 uscite corrispondono alle 10 parole chiave più le classi Silence e Unknown, e l'ultimo strato usa la Softmax per ottenere le probabilità delle classi.

Partendo dalla Figura 2.2, è possibile distinguere i vari strati della rete, che si ripetono uguali per diversi blocchi:

- L'**Input Layer** è caratterizzato da una matrice di feature di dimensioni $(51, 10, 1)$, ossia 51 frame temporali (si veda l'Equazione 2.1; Hello Edge ne usa 49 perché non applica il padding), 10 coefficienti MFCC, 1 canale, ossia l'audio trasformato in una "immagine" tempo-frequenza.

- **Conv2D (64, 10x4, 2x2)** è una convoluzione 2D classica con 64 filtri kernel di dimensione 10×4 (10 nella dimensione temporale e 4 in quella della frequenza) e stride 2×2 (il filtro si sposta di due posizioni sia nel tempo che in frequenza a ogni passo, invece di una sola come con stride unitario, il valore di default).

- **Batch Normalization** (BN) normalizza le attivazioni di ogni canale in modo che, sul mini-batch, abbiano media 0 e varianza 1, e poi applica un fattore di scala e uno spostamento appresi durante l'addestramento.

- **ReLU** (Rectified Linear Unit) è una funzione di attivazione non lineare, che assegna uno 0 per ogni input negativo, mentre mantiene gli input positivi associando all'output il valore di ingresso.

- **DepthwiseConv2D (1, 3x3, 1)** consiste in una convoluzione Depthwise in cui il "1" iniziale indica il depth multiplier (quanti filtri per canale, in questo caso un filtro per canale, il caso standard), 3×3 è la dimensione spaziale del kernel, l'ultimo 1 è lo stride. A differenza della Conv2D standard vista prima, qui ogni canale in ingresso viene convoluto separatamente con il proprio filtro 3×3, senza mescolare informazioni tra canali diversi. Con 64 canali in ingresso, questo strato usa 64 filtri 3×3 distinti, uno per canale, e produce ancora 64 canali in uscita (uno per filtro).

- **Conv2D (64, 1x1, 1)** è una convoluzione pointwise: per ogni singola posizione (tempo, frequenza), prende il vettore di 64 valori (uno per canale) e lo moltiplica per una matrice di pesi 64×64, producendo un nuovo vettore di 64 valori.

Il blocco DepthwiseConv → BN → ReLU → Conv 1×1 → BN → ReLU si ripete quattro volte di seguito, ciascuno mantenendo 64 canali in uscita; pertanto, la rete approfondisce l'estrazione di feature senza aumentare eccessivamente il costo computazionale.

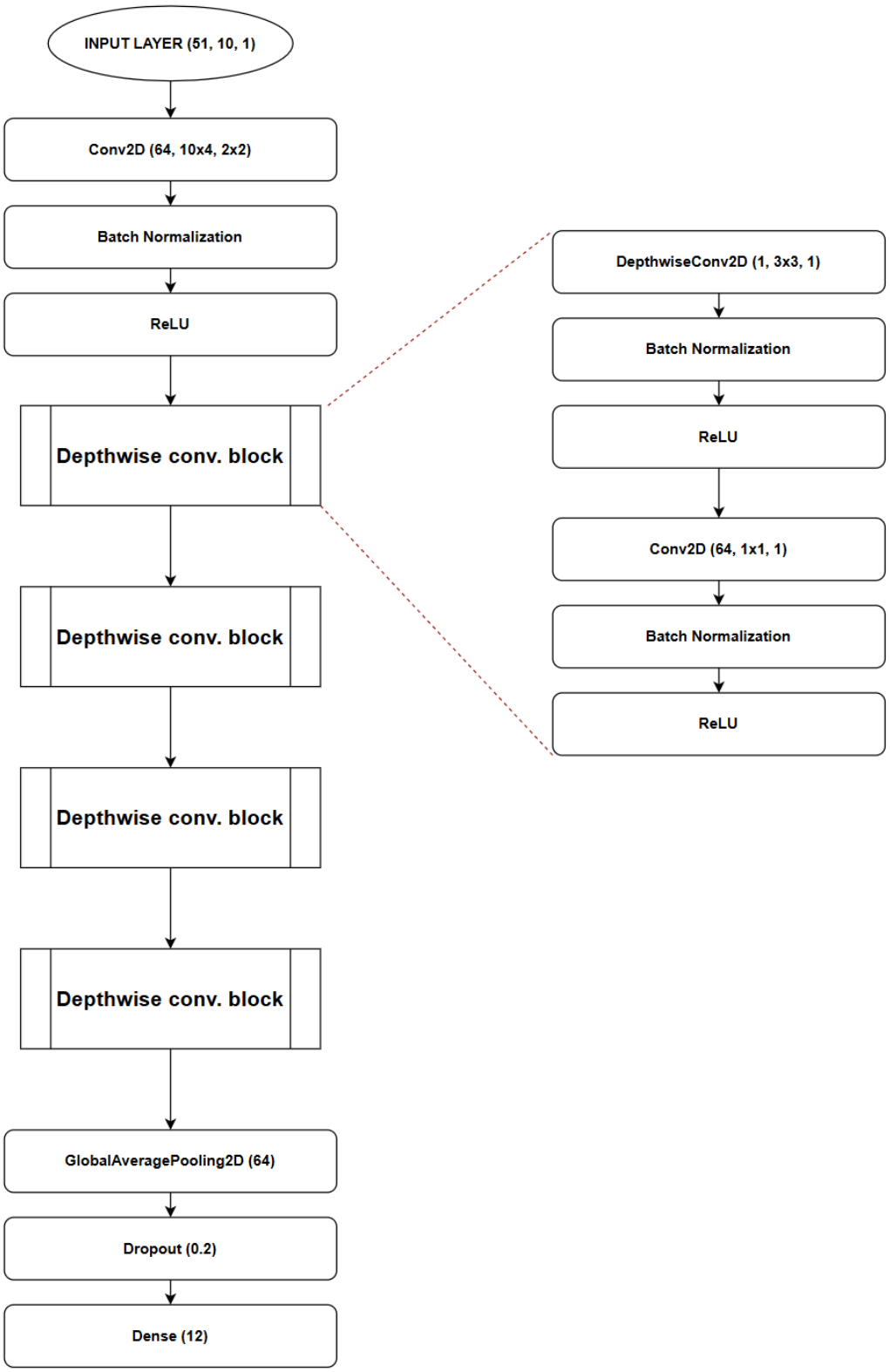



Figura 2.2: *Architettura della rete utilizzata.*

- **GlobalAveragePooling2D (64)**: dopo il quarto blocco, l'output è ancora una mappa 3D (dimensione_tempo × dimensione_frequenza × 64 canali). Questo strato calcola, per ciascuno dei 64 canali separatamente, la media di tutti i valori spaziali di quel canale, producendo un singolo numero per canale.
- **Dropout**: durante l'addestramento, questo strato spegne casualmente una percentuale di neuroni, con probabilità 0.2 (ovvero il 20% di probabilità di spegnimento) in questo progetto. In questo modo si costringe la rete a non fare un eccessivo affidamento su singoli neuroni specifici, riducendo l'overfitting, ossia il rischio che il modello "memorizzi" il training set invece di generalizzare.
- **Dense**: in questo strato, ogni neurone è collegato a tutti i neuroni dello strato precedente, in modo da combinare tutte le feature acquisite precedentemente. Lo strato finale ha 12 neuroni con attivazione Softmax, uno per classe, e restituisce la probabilità che il campione audio appartenga a ciascuna classe.

La combinazione depthwise + pointwise è esattamente ciò che rende la DS-CNN più efficiente (meno parametri, meno calcoli e minor tempo di inferenza) rispetto a una convoluzione 2D standard che farebbe tutto in un unico passaggio.

## 2.2 Dataset: Google Speech Commands Dataset (GSCD)

*Google Speech Commands Dataset Versione 2 (GSCD v.2)* [3, 4] è un dataset audio pubblico e su larga scala per il riconoscimento di parole chiave pronunciate, ampiamente utilizzato nella ricerca su *KWS*, classificazione di comandi vocali, strategie di data augmentation e modelli profondi efficienti; particolarmente adatto per modelli a basso ingombro ed efficienti dal punto di vista energetico. Il dataset è caratterizzato da 35 parole di comando, ognuna dalla durata di 1 s in formato WAV, e con suddivisioni fisse in training, validazione e test. Le clip sono campionate a 16 kHz. Delle 35 parole se ne usano 10 come parole chiave (yes, no, up, down, left, right, on, off, stop, go), mentre le altre 25 confluiscono nella classe Unknown, sottocampionata per bilanciarla. La classe Silence è ottenuta ritagliando clip da 1 s dai file di rumore di fondo. Si ottengono così 12 classi. Lo split ufficiale conta circa 37.000 campioni di training, 4.443 di validation e 4.888 di test.

## 2.3 Coefficienti MFCC per il riconoscimento vocale

Gli MFCC [5, 6] sono caratteristiche ampiamente utilizzate nei sistemi di riconoscimento automatico del parlato e del parlante. Possono essere definiti come un modo per rappresentare lo spettro di potenza di un suono, che aiuta le macchine a comprendere ed elaborare il linguaggio umano in modo più efficace, catturando le caratteristiche più significative del parlato e distinguendo parole e suoni differenti. Dal punto di vista teorico, gli MFCC sono coefficienti numerici che descrivono l'inviluppo

spettrale del segnale vocale. La loro estrazione avviene attraverso una sequenza di operazioni di elaborazione del segnale, finalizzate a ottenere una rappresentazione compatta ed efficace delle informazioni contenute nella voce. Le principali fasi del processo sono le seguenti:

- **Analisi del segnale**: il parlato è un segnale complesso, caratterizzato da variazioni di frequenza e ampiezza. Gli MFCC aiutano a scomporre questi segnali in componenti più semplici, che rappresentano la velocità e le caratteristiche delle variazioni delle onde sonore nel tempo.

- **Trasformazione in frequenza**: gli esseri umani non percepiscono le frequenze secondo una scala lineare. Per questo motivo, gli MFCC utilizzano la scala Mel, che approssima il comportamento del sistema uditivo umano, maggiormente sensibile alle variazioni delle basse frequenze rispetto a quelle delle alte frequenze.

- **Rappresentazione cepstrale**: dopo la trasformazione sulla scala Mel, si applica la DCT (passo 7 dell'elenco seguente) e si passa al dominio cepstrale, detto della *quefrency*, che non coincide con il dominio del tempo. Questa trasformazione si chiama *cepstrum*. Il *cepstrum* permette di separare le variazioni periodiche del segnale, associate all'altezza del suono (*pitch*), dalle variazioni più lente, associate al timbro.

L'estrazione delle feature per i modelli neurali prevede tipicamente spettrogrammi Mel. La scala Mel è stata progettata specificamente per imitare il modo in cui gli esseri umani percepiscono i suoni, in particolare per quanto riguarda la capacità di distinguere le differenze nell'altezza del suono (*pitch*). L'udito umano è più sensibile alle variazioni delle basse frequenze rispetto a variazioni equivalenti delle alte frequenze. Pertanto, la scala Mel tiene conto di questa caratteristica utilizzando una distribuzione lineare delle frequenze al di sotto dei 1000 Hz e una distribuzione logaritmica al di sopra dei 1000 Hz. Questa trasformazione consente di ottenere una rappresentazione dei segnali audio più coerente con la percezione umana, che è di natura non lineare. Nella regione lineare, quindi alle basse frequenze, inferiori a 1000 Hz, il nostro orecchio è in grado di rilevare piccole differenze nell'altezza del suono. La scala Mel riproduce questa sensibilità distribuendo linearmente tali frequenze. Ciò significa che una variazione della frequenza corrisponde direttamente a una variazione proporzionale sulla scala. Invece, nella regione logaritmica, ossia al di sopra dei 1000 Hz, il nostro orecchio è meno sensibile alle variazioni di frequenza. In questa regione, la scala Mel assume un andamento logaritmico, raggruppando più vicine le frequenze che, dal punto di vista percettivo, producono suoni simili. Di conseguenza, all'aumentare della frequenza, sono necessarie variazioni più ampie per ottenere una differenza percettiva analoga nell'altezza del suono.

Per calcolare i coefficienti MFCC, si seguono i seguenti passaggi:

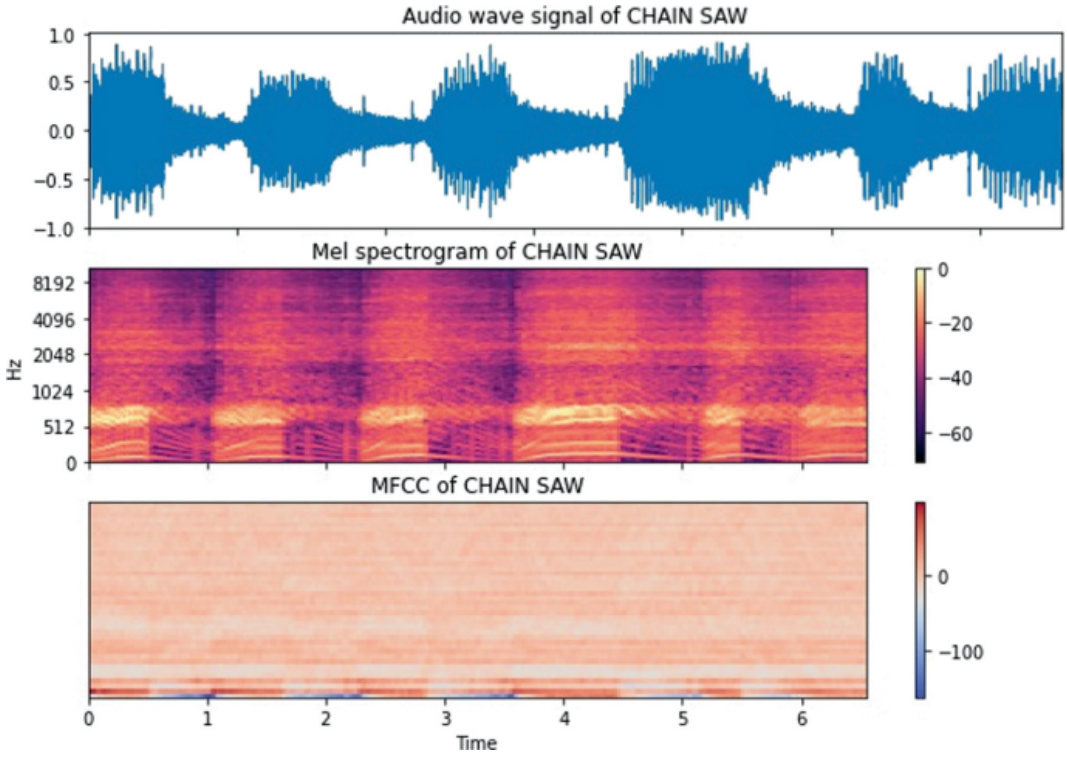



Figura 2.3: *Tre rappresentazioni dello stesso segnale audio, un suono di motosega. In alto: segnale audio nel tempo; al centro: spettrogramma Mel; in basso: MFCC.*

1. **Pre-enfasi del segnale**: si amplificano le alte frequenze per bilanciare lo spettro, poiché il parlato tende a presentare una minore energia alle alte frequenze, a causa delle caratteristiche fisiologiche del tratto vocale umano e delle proprietà di propagazione del suono. Questa operazione può contribuire a rendere più evidenti le caratteristiche rilevanti del parlato rispetto al rumore di fondo, facilitandone l'analisi.

2. **Framing**: il flusso continuo della voce viene suddiviso in segmenti più brevi, chiamati *frame* (finestre temporali), che durano generalmente tra i 20 e i 40 ms. In questo modo, è possibile catturare ed esaminare in modo più efficace le proprietà dinamiche del segnale vocale.

3. **Windowing**: si applica una finestra di Hamming per smussare i bordi di ciascun frame, riducendo le variazioni improvvise dell'ampiezza del segnale e minimizzando le discontinuità ai margini del frame.

4. **Fast Fourier Transform - FFT**: si converte ogni frame dal dominio del tempo al dominio della frequenza.

5. **Mel-filterbank**: il banco è costituito da un insieme di filtri, ciascuno dei quali è sintonizzato su uno specifico intervallo di frequenze, secondo la scala Mel. Il banco di filtri suddivide il risultato della FFT in queste bande, misurando il livello di energia di ciascuna di esse e mettendo in evidenza le frequenze più rilevanti per la percezione uditiva umana.

6. **Spettro Mel logaritmico**: si calcola il logaritmo dei valori in uscita dal banco di filtri, per riprodurre il modo in cui l'orecchio umano percepisce l'intensità sonora.

7. **Trasformata discreta del coseno - DCT**: si applica la trasformata discreta del coseno allo spettro Mel logaritmico per ottenere i coefficienti cepstrali in frequenza Mel (MFCC).

In conclusione, gli MFCC rappresentano un elemento fondamentale della tecnologia di riconoscimento vocale, poiché forniscono un metodo efficace e affidabile per rappresentare i segnali del parlato.

# Capitolo 3

# Svolgimento del progetto

## 3.1 Fase 1: Download in locale del Google Speech Commands

Come primo passo del progetto, iniziamo l'estrazione del dataset di Google Speech Commands (v0.02). Per farlo sfruttiamo urllib e tarfile, che permettono di scaricare e decomprimere l'archivio contenente il dataset (Figura 3.1).

Il dataset è composto da 36 cartelle: 35 cartelle, una per ciascuna parola chiave, ognuna contenente un numero di file variabile della durata di 1 secondo, più la cartella *_background_noise_* che contiene file audio di rumore di fondo. Le cartelle sono le seguenti:

- *background noise*
- *backward*
- *bed*
- *bird*
- *cat*
- *dog*
- *down*
- *eight*
- *five*
- *follow*
- *forward*
- *four*
- *go*
- *happy*
- *house*
- *learn*
- *left*
- *marvin*
- *nine*
- *no*
- *off*
- *on*
- *one*
- *right*
- *seven*
- *sheila*
- *six*
- *stop*
- *three*
- *tree*
- *two*
- *up*
- *visual*
- *wow*
- *yes*
- *zero*

## 3.2 Fase 2: Suddivisione del dataset in Training, Validation e Test

Una volta ottenuto il dataset in locale, procediamo con la suddivisione in tre sotto-dataset: Training, Validation e Test. Per farlo sfruttiamo uno script Python che scansiona il dataset, divide i file audio in base alla lista di split fornita da Google, per poi bilanciare le classi *Unknown* e *Silence* in base alla dimensione media delle classi target.

La classe *Unknown* deve semplicemente essere sottocampionata, in quanto racchiude 25 classi a sé stanti, e conterrebbe quindi troppi campioni rispetto alle classi target. La classe *Silence*, invece, deve essere generata ad hoc ritagliando clip di 1 secondo dai

```python
def main():
    # Verifichiamo l'esistenza della cartella di destinazione, e se non esiste la creiamo
    os.makedirs(DEST_DIR, exist_ok=True)

    # Verifichiamo l'esistenza dell'archivio, e se non esiste lo scarichiamo
    # Se l'archivio esiste già, non lo scarichiamo nuovamente
    archive_path = os.path.join(DEST_DIR, "speech_commands_v0.02.tar.gz")
    if os.path.exists(archive_path):
        print(f"L'archivio esiste già in: {archive_path}")
    else:
        print(f"Scarico il dataset in: {archive_path}")
        urllib.request.urlretrieve(DATA_URL, archive_path, reporthook=download_progress)
        print("\nDownload completato.")

    # Creiamo la cartella di destinazione se non esiste ed iniziamo ad estrarre l'archivio
    print(f"Estraggo l'archivio in: {DEST_DIR}")
    with tarfile.open(archive_path, "r:gz") as tar:
        tar.extractall(path=DEST_DIR)
    print("Estrazione completata.")

    # Stampiamo a video le cartelle keyword trovate e il numero di file in ciascuna
    print("\nCartelle keyword trovate:")
    for entry in sorted(os.listdir(DEST_DIR)):
        full_path = os.path.join(DEST_DIR, entry)
        if os.path.isdir(full_path):
            n_files = len(os.listdir(full_path))
            print(f"  {entry:20s} -> {n_files} file")

    # al termine dell'estrazione, stampiamo a video il path della cartella di destinazione
    print(f"\nDataset pronto in: {DEST_DIR}")
```

Figura 3.1: *Parte dello script di estrazione del dataset Google Speech Commands*

file audio già presenti nel dataset, ma troppo lunghi per essere utilizzati correttamente (le clip audio originali hanno una durata di circa 1 minuto).

Il risultato di questa fase è la creazione di tre file CSV, ognuno contenente due colonne: *filepath* e *label*, e un numero di righe pari al numero di campioni per split.

## 3.3 Fase 3: Estrazione delle feature MFCC dai file audio

Ottenuti i tre sotto-dataset, procediamo con l'estrazione delle feature MFCC dai file audio. Per fare ciò sfruttiamo la libreria *SpeechBrain*, che permette di caricare i file audio e calcolare le feature MFCC in maniera semplice ed efficiente.

Per effettuare l'estrazione, andiamo prima a leggere i file csv generati nella fase precedente, e per ogni riga carichiamo il corrispondente file audio, ad eccezione dei file della classe *Silence*, per i quali ritagliamo un secondo casuale dal file audio originale.

Una volta ottenuta la clip audio, estraiamo le feature MFCC e le salviamo in un file .npz, contenente due array: uno per le feature, ed uno per le label. Il risultato dell'estrazione può essere visualizzato in Figura 3.2.

L'estrazione è stata effettuata prima con una FFT, per poi dividere lo spettro mediante 40 filtri Mel con una risoluzione maggiore alle basse frequenze. Successivamente, è stata compressa la dinamica mediante un logaritmo (che approssima la percezione logaritmica dell'intensità sonora da parte dell'orecchio umano), ed infine una DCT (Discrete Cosine Transform), di cui vengono tenuti i primi 10 coefficienti.

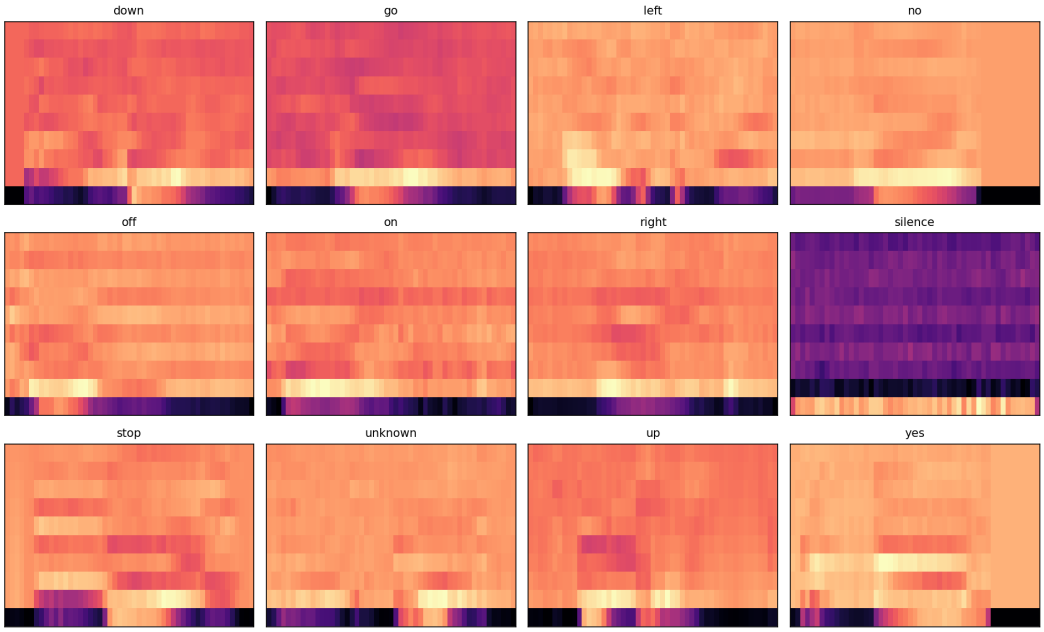



Figura 3.2: *Visualizzazione di alcune feature MFCC estratte per ogni classe target*

Ogni grafico è composto da due assi principali: l'asse X che rappresenta il frame temporale, in cui ogni campione corrisponde a 20 ms di audio, per un totale di 51 frame (1 secondo); mentre nell'asse Y sono rappresentati i coefficienti MFCC da 0 a 9, con le loro funzioni corrispondenti:

- **MFCC 0**: Rappresenta il livello complessivo di energia di tale frame (intensità sonora). Non contiene informazioni su timbro o frequenza;
- **MFCC 1**: Rappresenta il bilanciamento tra basse e alte frequenze (la pendenza dello spettro), e differenzia dei suoni "cupi" da suoni più chiari come la "$s$";
- **MFCC 2**: Rappresenta l'energia agli estremi dello spettro rispetto al centro;
- **MFCC 3-4**: Rappresentano la struttura media dello spettro, permettendo di riconoscere le risonanze che caratterizzano le vocali;
- **MFCC 5-9**: Rappresentano i dettagli spettrali, e sono meno interpretabili e più sensibili al rumore, ma permettono di distinguere meglio le consonanti.

In generale, le prime feature trasportano l'informazione più importante e robusta per il riconoscimento vocale, mentre le più alte i dettagli. La nostra rete andrà a riconoscere quali pattern tra coefficienti e frame distinguono le varie parole, in maniera analoga a come una rete CNN classica riesce a riconoscere delle feature da immagini.

## 3.4 Fase 4: Creazione della rete DS-CNN

La rete che abbiamo utilizzato è una DS-CNN, in quanto è risultata la scelta migliore come rapporto accuracy/occupazione in memoria. Per realizzarla abbiamo adottato la struttura illustrata in Hello Edge [2] alla tabella 7.

Una volta realizzata, la rete può mostrare un sommario della propria architettura, della dimensione e del numero di parametri (Figura 3.3).

```python
def build_ds_cnn(
  input_shape,
  num_classes: int,
  num_filters: int = 64,
  n_blocks: int = 4,
  first_kernel=(10, 4),
  first_strides=(2, 2),
  dropout: float = 0.2,
  name: str = "ds_cnn_s",
):
  inputs = layers.Input(shape=input_shape, name="mfcc_input")
  # Primo layer: convoluzione "normale" (non depthwise-separable)
  x = layers.Conv2D(num_filters, first_kernel, strides=first_strides, padding="same")(inputs)
  x = layers.BatchNormalization()(x)
  x = layers.ReLU()(x)
  # Blocchi depthwise-separable: il cuore della DS-CNN, quello che la
  # rende leggera rispetto a una CNN "piena" equivalente

  for i in range(n_blocks):
    x = layers.DepthwiseConv2D((3, 3), padding="same", name=f"dw_conv_{i}")(x)
    x = layers.BatchNormalization()(x)
    x = layers.ReLU()(x)
    x = layers.Conv2D(num_filters, (1, 1), padding="same", name=f"pw_conv_{i}")(x)
    x = layers.BatchNormalization()(x)
    x = layers.ReLU()(x)
  x = layers.GlobalAveragePooling2D()(x)
  if dropout > 0:
    x = layers.Dropout(dropout)(x)
  outputs = layers.Dense(num_classes, activation="softmax", name="predictions")(x)
  return models.Model(inputs, outputs, name=name)


if __name__ == "__main__":
  model = build_ds_cnn(INPUT_SHAPE, NUM_CLASSES)
  model.summary()
```

Figura 3.3: *Architettura della rete DS-CNN utilizzata*

La rete, formata da quattro blocchi depthwise separable, conta 24.908 parametri, con un'occupazione in memoria di 97.30 KB (in precisione float32, quindi non quantizzata).

In ingresso alla rete forniamo un input formato da 51 frame, 10 parametri MFCC ed 1 canale, per poi effettuare un output softmax su 12 classi: le 10 parole chiave target (*yes*, *no*, *up*, *down*, *left*, *right*, *on*, *off*, *stop*, *go*), più le classi *Unknown* e *Silence*.

La struttura è composta da tre parti:

- **Layer iniziale**: una convoluzione classica con 64 filtri, kernel $10 \times 4$, stride $2 \times 2$ e padding *same*, seguita da Batch Normalization e ReLU, che porta la mappa di feature a dimensione $26 \times 5 \times 64$;

- **Quattro blocchi depthwise separable**: ognuno composto da una convoluzione depthwise $3 \times 3$ e da una convoluzione pointwise $1 \times 1$ con 64 filtri, entrambe seguite da Batch Normalization e ReLU;
- **Parte finale**: un Global Average Pooling, un Dropout con probabilità 0.2 per ridurre l'overfitting, ed un layer Dense con 12 neuroni e attivazione softmax.

Il vantaggio della convoluzione depthwise separable sta nella divisione dell'operazione in due passaggi: la convoluzione depthwise filtra ogni canale in maniera indipendente, mentre la pointwise combina i canali tra loro.

La rete è facilmente modellabile, ed è stata poi modificata per testare un sistema alleggerito a due blocchi, composto da 14.284 parametri, con un'occupazione in memoria di 55.80 KB (float32). Il confronto tra le due versioni è riportato in Tabella 3.3.

## 3.5 Fase 5: Addestramento

### 3.5.1 Configurazione dell'ambiente di sviluppo

Per ridurre i tempi di addestramento, la rete è stata addestrata in locale sfruttando una RTX 3070 Ti. Per poter utilizzare la GPU con TensorFlow su Windows, è necessario considerare che, a partire dalla versione 2.11, TensorFlow non supporta più l'accelerazione GPU nativa su Windows, pertanto le strade per continuare diventano due: o continuare ad utilizzare Windows, utilizzando la versione più recente che supportasse ancora l'accelerazione GPU, oppure spostarsi su Linux mediante macchina virtuale.

Per questo progetto abbiamo deciso di rimanere in ambiente Windows, per cui l'ultima versione utilizzabile è la 2.10. Tale versione richiede a sua volta versioni specifiche delle librerie NVIDIA:

| **Componente** | **Versione** |
|---|---|
| Python | 3.9 |
| TensorFlow | 2.10 (`tensorflow<2.11`) |
| CUDA Toolkit | 11.2 |
| cuDNN | 8.1.0 |
| NumPy | 1.24 (`numpy<2`) |

Tabella 3.1: *Versioni dei componenti necessarie per l'addestramento su GPU locale in ambiente Windows*

Per isolare queste dipendenze dal resto del sistema è stato utilizzato Miniconda, creando un ambiente virtuale dedicato. Il vantaggio principale di conda è che permette di installare CUDA Toolkit e cuDNN direttamente all'interno dell'ambiente, senza doverli installare a livello di sistema: l'unico requisito esterno è il driver NVIDIA

della scheda video. Si noti inoltre che TensorFlow 2.10 è stato compilato con NumPy 1.24, ed è quindi necessario evitare l'installazione di NumPy 2, che causerebbe errori in fase di importazione.

Prima di tutto, è necessaria l'installazione dell'ultima versione di Microsoft Visual C++ Redistributable, l'installazione di Miniconda per Windows e l'avvio tramite Anaconda PowerShell Prompt.

L'ambiente è stato creato mediante i seguenti comandi:

```
conda create -n tf-gpu python=3.9
conda activate tf-gpu
python -m pip install --upgrade pip
conda install -c conda-forge cudatoolkit=11.2 cudnn=8.1.0
pip install "tensorflow<2.11" "numpy<2"
```

Una volta configurato l'ambiente, la corretta rilevazione della GPU può essere verificata mediante la funzione `tf.config.list_physical_devices("GPU")`, richiamata anche all'avvio dello script di addestramento.

### 3.5.2 Parametri di addestramento

Per l'addestramento sono stati adottati gli stessi iperparametri indicati in Hello Edge [2]:

- **Ottimizzatore**: Adam;
- **Funzione di costo**: Sparse Categorical Cross-Entropy, in quanto le label sono rappresentate come indici interi delle 12 classi;
- **Batch size**: 100 campioni;
- **Numero di iterazioni**: 20.000;
- **Learning rate**: pari a $5 \cdot 10^{-4}$ per la prima metà delle iterazioni, e ridotto a $10^{-4}$ per la seconda metà.

La funzione di costo non viene esplicitamente indicata nel paper, ma risulta la scelta più diffusa. Tale funzione misura quanto la probabilità assegnata alla classe corretta sia lontana da 1. Questo valore è quello che Adam punta a minimizzare ad ogni batch. Poiché Keras ragiona in termini di epoche e non di iterazioni, il numero di iterazioni indicato nel paper è stato convertito in epoche in base alla dimensione effettiva del training set:

$$N_{epoche} = \frac{N_{iterazioni} \cdot \text{batch size}}{N_{train}} = \frac{20.000 \cdot 100}{N_{train}} \approx 54 \tag{3.1}$$

dove $N_{train}$ è il numero di campioni del training set (circa 37.000). La riduzione del learning rate avviene quindi a partire dall'epoca 28, ed è gestita tramite il callback

*LearningRateScheduler*. Ridurre il learning rate nella seconda parte dell'addestramento permette all'ottimizzatore di compiere passi più piccoli una volta raggiunta una zona vicina al minimo, stabilizzando la convergenza e riducendo il rischio di overfitting.

Ad ogni epoca la rete viene valutata sul validation set, e tramite il callback *ModelCheckpoint* viene salvato il modello con la migliore accuratezza di validazione. Per garantire la riproducibilità dei risultati è stato fissato un seed casuale per Python, NumPy e TensorFlow, mentre ad ogni epoca i campioni del training set vengono rimescolati. Non è stata applicata data augmentation.

### 3.5.3 Risultati dell'addestramento

L'andamento dell'addestramento della rete originale è mostrato in Figura 3.4, mentre in Tabella 3.2 sono riportati i valori in alcune epoche significative.

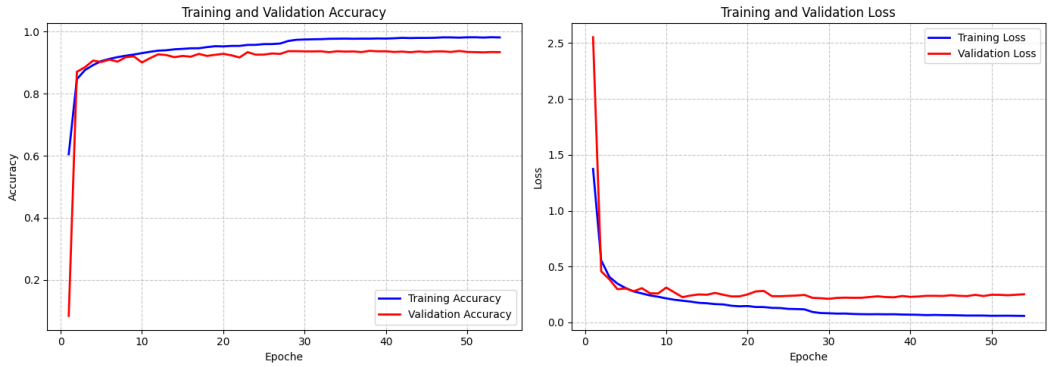



Figura 3.4: *Accuratezza e loss su training e validation set della rete originale a quattro blocchi*

| Epoca | Train acc. | Train loss | Val acc. | Val loss |
|---|---|---|---|---|
| 1 | 60.28% | 1.377 | 9.18% | 2.580 |
| 5 | 90.49% | 0.306 | 90.50% | 0.293 |
| 10 | 93.25% | 0.210 | 88.84% | 0.352 |
| 20 | 95.35% | 0.142 | 93.00% | 0.236 |
| 27 | 96.23% | 0.115 | 92.19% | 0.269 |
| 28 | 97.14% | 0.090 | 93.68% | 0.212 |
| **30** | **97.46%** | **0.081** | **93.92%** | **0.209** |
| 40 | 97.85% | 0.069 | 93.68% | 0.219 |
| 54 | 98.14% | 0.059 | 93.41% | 0.240 |

Tabella 3.2: *Accuratezza e loss su training e validation set in alcune epoche significative. In grassetto il modello salvato.*

Alla prima epoca l'accuratezza di validazione è molto bassa (9.18%, vicina al valore di una classificazione casuale, $1/12 =\sim 8.3\%$), ma già dalla quinta epoca supera

il 90%. Fino all'epoca 27 la validazione oscilla tra l'88% ed il 93%, a causa di un learning rate ancora elevato. Con la riduzione del learning rate all'epoca 28 si osserva un miglioramento sia in accuratezza che in loss, ed il modello migliore viene raggiunto all'epoca 30, con un'accuratezza di validazione del **93.92%**.

Nelle epoche successive l'accuratezza sul training set continua a crescere fino al 98.14%, mentre la loss di validazione risale da 0.209 a 0.240: la rete sta quindi iniziando ad andare in overfitting. Il modello salvato rimane comunque quello dell'epoca 30, che non risente di questo effetto.

### 3.5.4 Prove con architetture alternative

A causa di questo piccolo overfitting, e sfruttando la modularità della rete, abbiamo effettuato ulteriori prove variando il numero di blocchi depthwise e la probabilità di dropout, mantenendo invariati tutti gli altri parametri di addestramento. L'obiettivo di tali prove è stato quello di ridurre la capacità di apprendimento della rete in maniera diretta, per rallentare l'apprendimento ed evitare che il modello "memorizzasse" il dataset.

Abbiamo inoltre provato una rete molto più pesante (dodici blocchi), per verificare se fosse proprio il numero di blocchi a determinare la velocità dell'apprendimento.

Le configurazioni testate sono:

- **Rete originale con dropout 0.5**: stessa struttura a quattro blocchi, con uno spegnimento più forte dei neuroni di attivazione;
- **Rete a due blocchi**: versione alleggerita, mantenendo un dropout di 0.2;
- **Rete a due blocchi con dropout 0.7**: versione alleggerita con una regolarizzazione molto forte;
- **Rete a dodici blocchi**: versione più pesante, con dropout 0.2.

Gli andamenti dei rispettivi addestramenti sono riportati nelle figure 3.5–3.8.

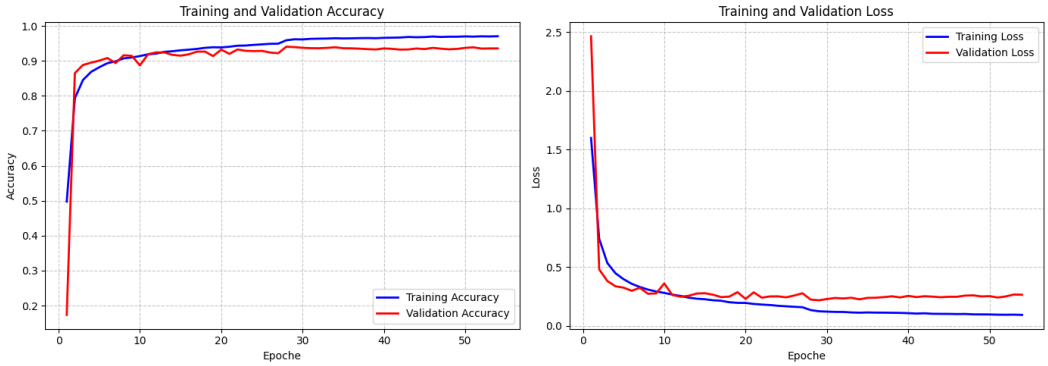



Figura 3.5: *Accuratezza e loss della rete a quattro blocchi con dropout 0.5*

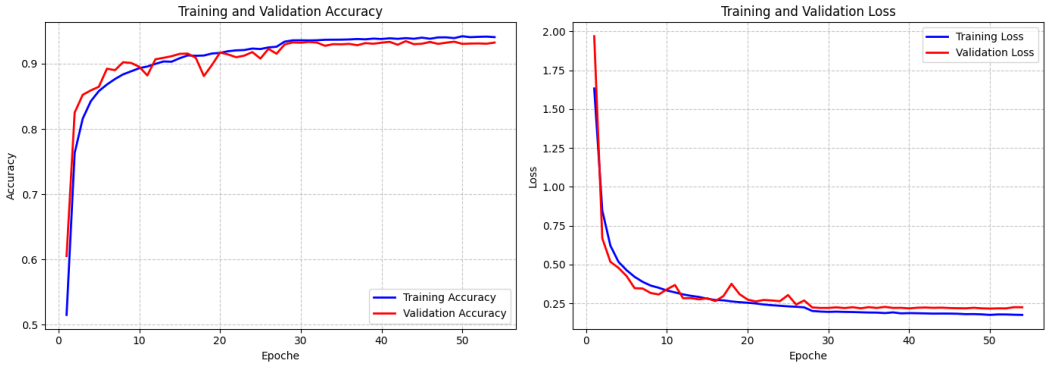



Figura 3.6: *Accuratezza e loss della rete a due blocchi con dropout 0.2*

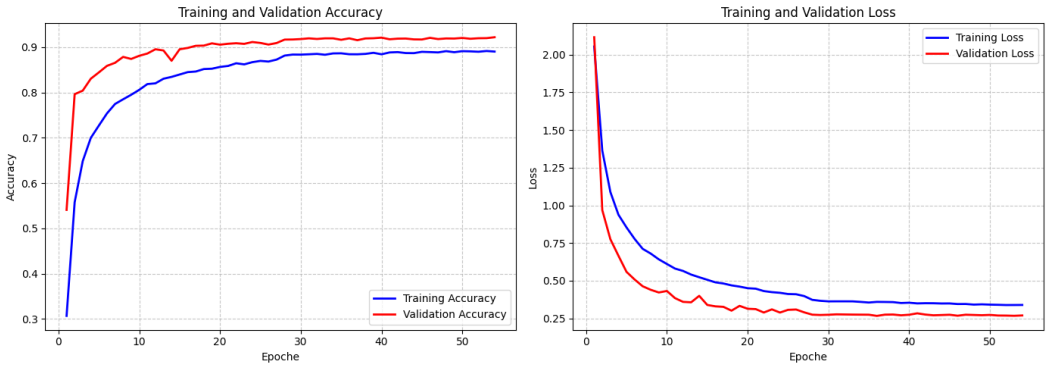



Figura 3.7: *Accuratezza e loss della rete a due blocchi con dropout 0.7*

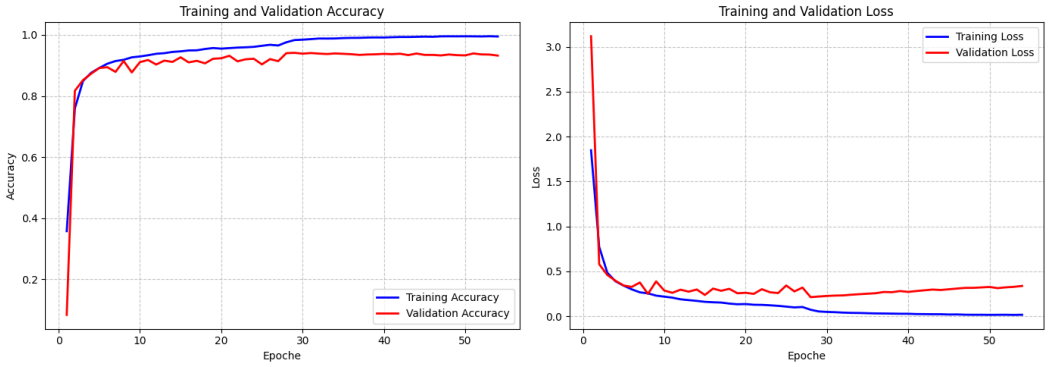



Figura 3.8: *Accuratezza e loss della rete a dodici blocchi con dropout 0.2*

I risultati delle cinque configurazioni sono riassunti in Tabella 3.3, dove l'accuratezza è calcolata sull'intero validation set (4.443 campioni).

| **Configurazione** | **Dropout** | **Parametri** | **Memoria (float32)** | **Val acc.** |
|---|---|---|---|---|
| 4 blocchi (originale) | 0.2 | 24.908 | 97.30 KiB | 93.92% |
| 4 blocchi | 0.5 | 24.908 | 97.30 KiB | 94.04% |
| 2 blocchi | 0.2 | 14.284 | 55.80 KiB | 93.47% |
| 2 blocchi | 0.7 | 14.284 | 55.80 KiB | 92.21% |
| 12 blocchi | 0.2 | 67.404 | 263.30 KiB | 94.13% |

Tabella 3.3: *Confronto tra le architetture testate*

Dal confronto emergono le seguenti osservazioni:

- **Dropout 0.5**: l'accuratezza sul training set si ferma intorno al 97%, riducendo la distanza dalla curva di validazione. Il guadagno in accuratezza di validazione è però minimo (+0.23%), a parità di occupazione in memoria;
- **Due blocchi**: le curve di training e validazione rimangono molto vicine per tutto l'addestramento, e la loss di validazione non risale nelle ultime epoche: l'overfitting è quindi molto ridotto. A fronte di una perdita di appena lo 0.34% di accuratezza, i parametri si riducono del 43%;
- **Due blocchi con dropout 0.7**: l'accuratezza di validazione rimane sempre superiore a quella di training, poiché il dropout è attivo solo durante l'addestramento. Essendo lo spegnimento dei neuroni molto spinto, la rete va in underfitting, ottenendo il risultato peggiore (92.21%) e confermando l'ipotesi per cui il miglioramento dell'overfitting non avvenisse prettamente dall'aumento del valore di Dropout;
- **Dodici blocchi**: l'accuratezza sul training set raggiunge quasi il 100%, mentre la loss di validazione cresce in maniera costante dopo la riduzione del learning rate, passando da circa 0.21 a 0.33, per cui la rete è evidentemente in overfitting. Pur ottenendo l'accuratezza più alta (94.13%), il miglioramento rispetto alla rete originale è di appena lo 0.32%, a fronte di un numero di parametri quasi triplicato.

In tutte le configurazioni l'accuratezza di validazione si assesta tra il 92% ed il 94%: aumentare il numero di blocchi Depthwise della rete non porta quindi benefici significativi, mentre ne aumenta notevolmente l'occupazione in memoria. Per questo motivo, per la validazione su scheda sono state mantenute solo la rete originale a quattro blocchi e la rete a due blocchi con dropout 0.2, che rappresentano rispettivamente il riferimento del paper ed il miglior compromesso tra accuratezza ed occupazione.

### 3.5.5 Matrici di confusione

Le matrici di confusione sul validation set delle due reti selezionate sono riportate nelle figure 3.9 e 3.10.

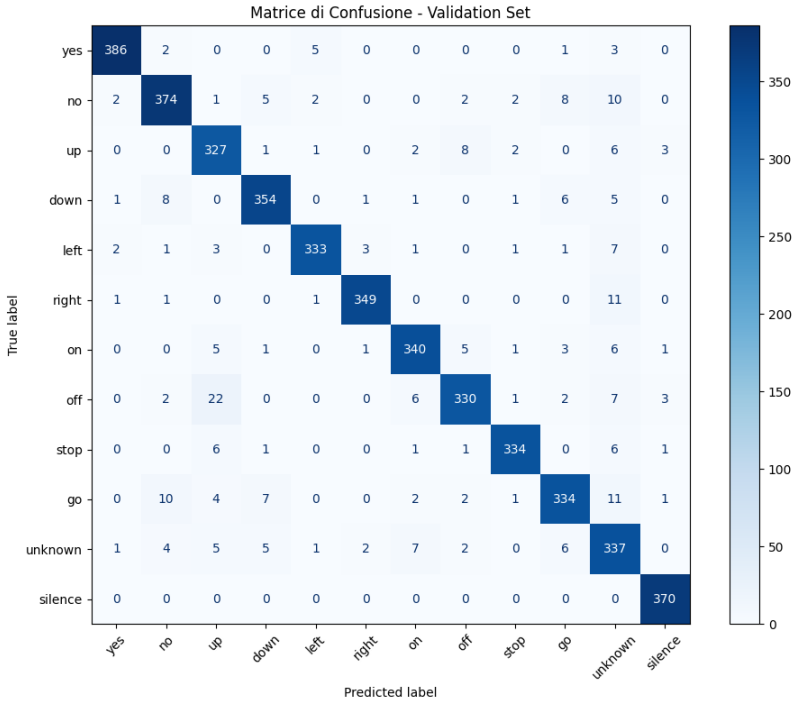



Figura 3.9: *Matrice di confusione sul validation set della rete originale a quattro blocchi*

Le due matrici presentano un comportamento molto simile, e gli errori più frequenti sono comuni ad entrambe le reti:

- **off** → **up**: è l'errore più frequente in assoluto (22 campioni per la rete originale, 29 per quella a due blocchi). Le due parole hanno una struttura fonetica simile, con una vocale breve seguita da una consonante sorda;
- **go** ↔ **no**: le due parole differiscono solo per la consonante iniziale, e condividono la stessa vocale finale;
- **go** → **down** e **down** → **no**: anche in questo caso si tratta di parole con suoni vocalici simili;
- **Unknown**: è la classe che raccoglie più errori in entrambe le direzioni, in quanto contiene 25 parole diverse, alcune delle quali foneticamente vicine alle parole target.

Le classi con l'accuratezza più bassa sono infatti *off* (88.5% e 87.4%), *go* (89.8% e 88.2%) e *Unknown* (91.1% e 85.9%), mentre la classe *Silence* viene riconosciuta

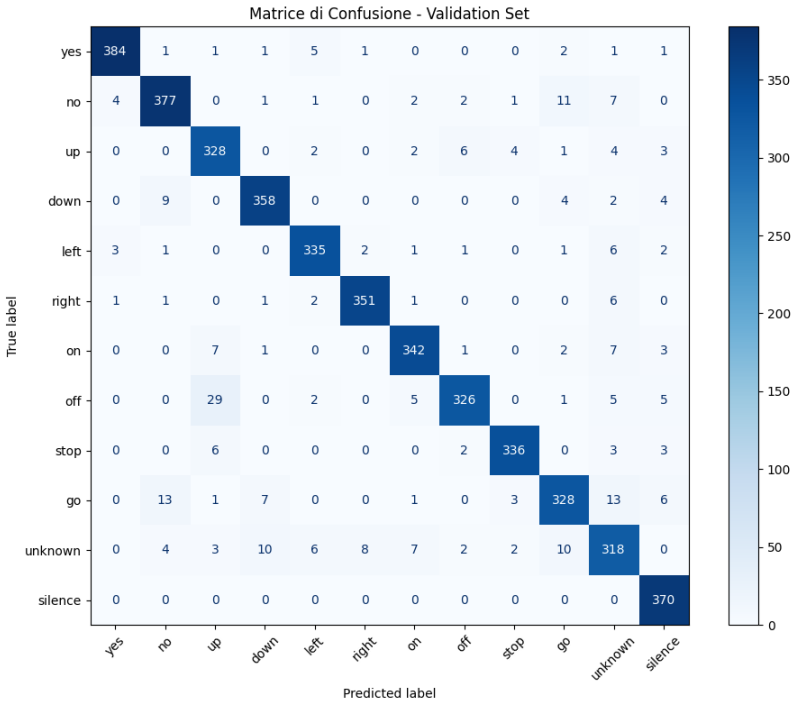



Figura 3.10: *Matrice di confusione sul validation set della rete a due blocchi*

correttamente nel 100% dei casi da entrambe le reti. Quest'ultimo risultato è dovuto al fatto che le clip di silenzio, essendo ricavate da rumore di fondo, sono molto diverse dal parlato, e sono inoltre estratte dagli stessi sei file audio per tutti e tre gli split.

## 3.6 Fase 6: Conversione del modello in tflite

Il modello ottenuto dall'addestramento è salvato nel formato originnale di Keras (*.keras*), che non può essere caricato direttamente su un microcontrollore. Per poterlo importare in STM32Cube AI Studio, che si occupa di generare il codice C ottimizzato per la scheda, il modello deve essere prima convertito nel formato *TensorFlow Lite* (*.tflite*), un formato compatto pensato appositamente per l'inferenza su dispositivi embedded.

La conversione viene effettuata tramite il *TFLiteConverter* di TensorFlow, a partire dal modello migliore salvato dal ModelCheckpoint nella fase precedente. Lo script genera due versioni del modello:

- **Modello float32**: una conversione diretta, in cui pesi e calcoli rimangono in virgola mobile a 32 bit. L'occupazione in memoria è invariata rispetto al modello Keras, e viene utilizzato come riferimento;
- **Modello int8**: una versione quantizzata, in cui pesi, attivazioni, ingresso ed uscita sono rappresentati come interi a 8 bit.

La quantizzazione int8 è quella più adatta al microcontrollore, in quanto riduce di circa quattro volte l'occupazione in memoria dei pesi, e permette di sostituire le operazioni in virgola mobile con operazioni intere, più veloci e meno dispendiose dal punto di vista energetico.

Per i pesi questi valori interi possono essere ricavati direttamente dai loro valori float, mentre per le attivazioni è necessario conoscere l'intervallo di valori che assumono durante l'inferenza. Per questo motivo si effettua una *post-training quantization* con calibrazione: al convertitore viene fornito un *representative dataset*, composto dai primi 100 campioni del validation set (che, essendo il file già mescolato, contengono esempi di tutte le 12 classi), che vengono fatti passare uno alla volta attraverso la rete per registrare i valori minimi e massimi raggiunti da ogni layer.

Infine, la conversione viene forzata ad utilizzare esclusivamente operazioni intere, ed anche l'ingresso e l'uscita della rete vengono impostati in int8.

## 3.7 Fase 7: Preparazione dei dati di validazione in formato NPY

Per validare il modello, STM32Cube AI richiede come dati di ingresso e come uscite attese dei file *.npy*, mentre i nostri dataset sono file .npz con due array, *features* e *labels*, dove le label sono ancora stringhe. Con lo script *Conversione_npy.py* carichiamo l'.npz, convertiamo in float32 e salviamo con *np.save*, ottenendo tensori di forma $(N, 51, 10, 1)$.

Per le uscite attese, lo script *Test_Conversion_Labels.py* carica le label stringa del test set e le mappa negli indici interi, usando lo stesso ordine *LABELS* del training (*yes*, *no*, *up*, *down*, *left*, *right*, *on*, *off*, *stop*, *go*, *unknown*, *silence*). Gli indici vengono poi trasformati in vettori e salvati in *test_outputs_expected.npy*:

```
label_to_idx = {lbl: i for i, lbl in enumerate(LABELS)}
y_test_int = np.array([label_to_idx[l] for l in y_test_str])
y_test_onehot = np.eye(NUM_CLASSES, dtype=np.float32)[y_test_int]
np.save(out_path, y_test_onehot)
```

I file usati per la validazione sono *test_inputs_only.npy*, di forma $(4888, 51, 10, 1)$, e *test_outputs_expected.npy*, di forma $(4888, 12)$, entrambi in float32: il test set è quindi composto da 4.888 campioni. Gli ingressi restano in virgola mobile perché è lo strumento a quantizzarli in int8, utilizzando la scala e lo zero point salvati nel modello, come anticipato alla fine della fase precedente.

## 3.8 Fase 8: Analisi del modello con STM32Cube AI Studio

Per l'analisi utilizziamo STM32Cube AI Studio con il motore ST Edge AI Core v4.0.1, selezionando come target la famiglia STM32F446RE. Il modello caricato è *ds_cnn_s_int8.tflite*, ovvero la nostra rete quantizzata. L'analisi è stata effettuata sia per la rete a due blocchi sia per la rete standard a quattro blocchi; i risultati sono riportati in Tabella 3.4.

| | **Rete a 2 blocchi** | **Rete a 4 blocchi** |
|---|---|---|
| Parametri | 13.004 | 22.604 |
| MACC | 1.557.120 | 2.772.096 |
| Pesi in flash | 14.000 B (13.67 KiB) | 24.368 B (23.80 KiB) |
| Attivazioni in RAM | 18.784 B (18.34 KiB) | 18.784 B (18.34 KiB) |
| Flash totale (pesi + runtime) | 41.190 B | 53.396 B |
| RAM totale | 19.088 B | 19.088 B |

Tabella 3.4: *Risultati dell'analisi con STM32Cube AI Studio delle due reti quantizzate*

In entrambi i casi i pesi occupano circa il 73% in meno rispetto al modello float (.keras) ($\sim$52 KB e $\sim$90 KB): 38 KB in meno per la rete a due blocchi e 66 KB in

meno per quella a quattro. I parametri sono inoltre meno di quelli contati da Keras (14.284 e 24.908), perché lo strumento unisce i layer di Batch Normalization nei pesi delle convoluzioni.

La RAM per le attivazioni è identica nelle due reti, perché ogni blocco depthwise separable mantiene la stessa mappa di feature $26 \times 5 \times 64$ (8.320 byte in int8), e il picco di memoria non dipende quindi dal numero di blocchi. Entrambe le reti stanno ampiamente nelle risorse dell'STM32F446, che dispone di 512 KB di flash e 128 KB di RAM. Le figure 3.11 e 3.12 riportano il riepilogo prodotto dal programma per le due reti: la rete a due blocchi usa 14 KB di flash e 18 KB di RAM, quella a quattro blocchi 24 KB di flash con la stessa RAM. Nella parte bassa di ciascuna figura compaiono anche le metriche di validazione, discusse nella fase successiva.

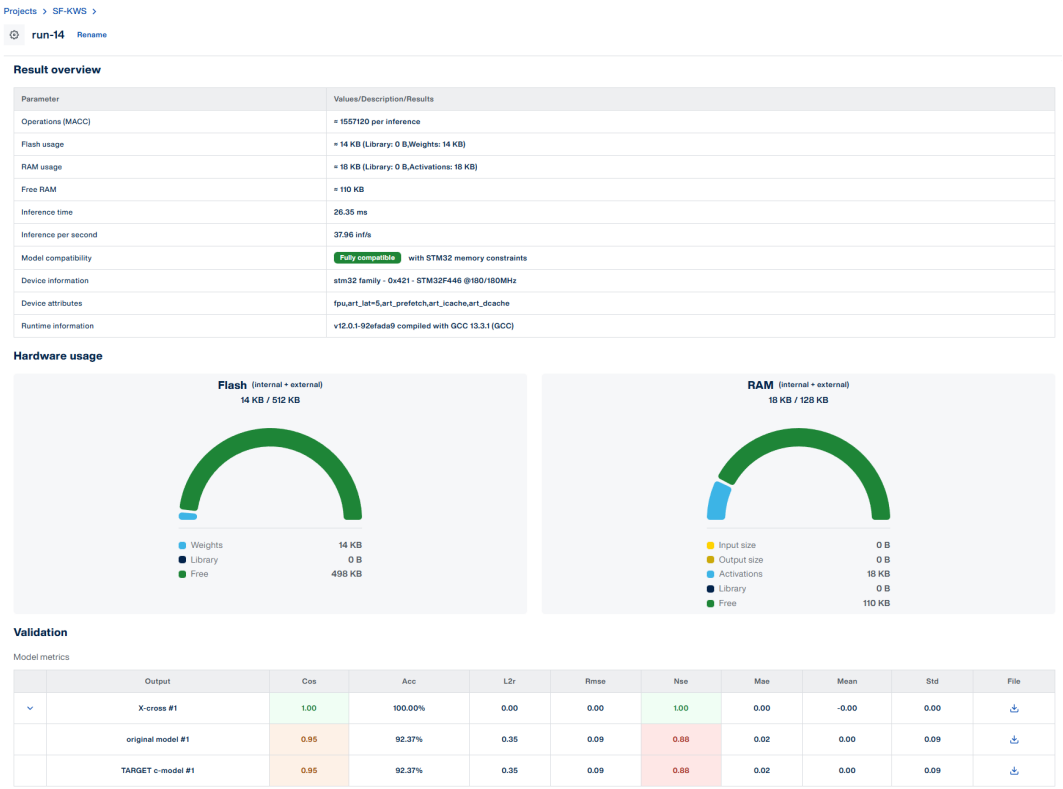



Figura 3.11: *Riepilogo di STM32Cube AI Studio per la rete a 2 blocchi: occupazione di flash e RAM e metriche di validazione*

## 3.9 Fase 9: Validazione on desktop e on target

Con la validazione on desktop il codice C generato viene eseguito sul PC e confrontato con il modello TFLite di riferimento, per verificare che la traduzione in C sia corretta prima di passare alla scheda. Con la validazione on target, invece, STM32Cube AI Studio genera e compila un firmware di validazione, lo programma

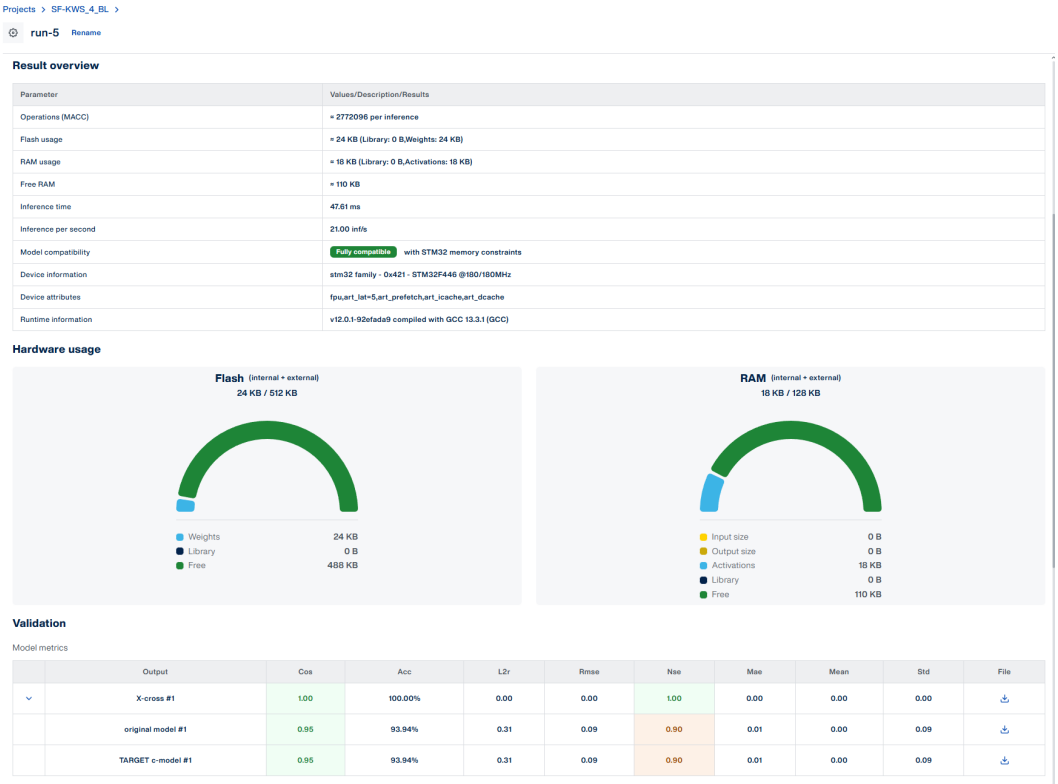
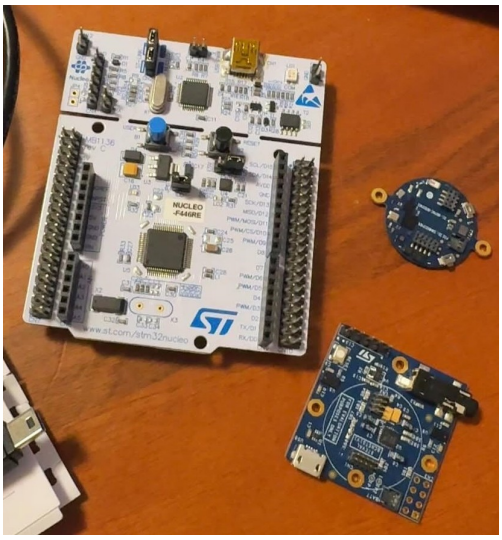


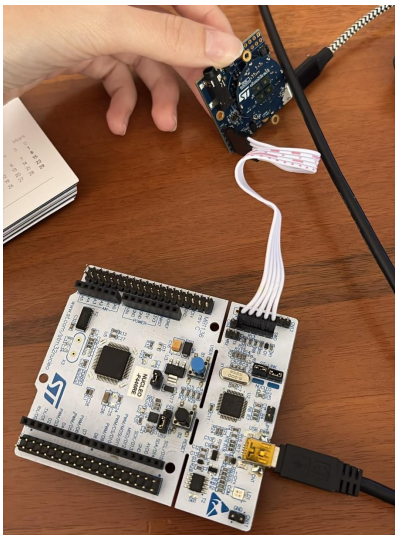

Figura 3.12: *Riepilogo di STM32Cube AI Studio per la rete a 4 blocchi: occupazione di flash e RAM e metriche di validazione*

sul microcontrollore e gli invia i 4.888 campioni di test via seriale (UART a 115200 baud), ricevendo le uscite e misurando tempi e cicli di CPU.

La piattaforma prevista era il kit BlueCoin, con microcontrollore STM32F446 a 180 MHz, programmato via SWD sfruttando il programmatore ST-LINK integrato in una scheda di sviluppo Nucleo (Figura 3.13). Alcune validazioni on target, però, non sono andate a buon fine. Abbiamo scoperto che la programmazione via SWD avveniva correttamente sul BlueCoin, mentre la comunicazione di validazione passava per la UART della porta seriale virtuale dell'ST-LINK, che sulla Nucleo è collegata al microcontrollore della Nucleo stessa. Il firmware veniva quindi caricato sul BlueCoin, ma lo scambio dei dati avveniva con la Nucleo. Le due schede montano lo stesso microcontrollore (famiglia STM32F446, stesso device ID 0x421, stessa frequenza di 180 MHz), ed è per questo che lo strumento non segnalava alcuna anomalia.



(a) *Le schede utilizzate: Nucleo-F446RE, BlueCoin e CoinStation*



(b) *Configurazione di programmazione: un cavo piatto collega la CoinStation al connettore SWD dell'ST-LINK sulla Nucleo, che è alimentata via USB*

Figura 3.13: *Hardware usato per la validazione on target*

Le validazioni finali sono state quindi eseguite direttamente sulla Nucleo-F446RE (STM32F446RET6). Avendo lo stesso microcontrollore e la stessa frequenza del BlueCoin, i tempi di inferenza e l'occupazione di memoria misurati sono rappresentativi di quelli che si otterrebbero su quest'ultimo. La configurazione riportata dallo strumento prevede STM32F446. I risultati sono riportati in Tabella 3.5.

Le metriche RMSE, MAE, e similarità coseno sono calcolate tra le uscite della rete e le uscite attese: le prime due misurano l'errore (quadratico medio, assoluto medio), mentre la similarità coseno misura quanto i due vettori siano allineati. Il modello di riferimento ottiene esattamente la stessa accuratezza della rete sulla scheda in entrambi i casi, e il confronto incrociato tra riferimento e codice C sulla

| | **Rete a 2 blocchi** | **Rete a 4 blocchi** |
|---|---|---|
| Accuratezza sul test set | 92.37% | 93.94% |
| RMSE | 0.0943 | 0.0869 |
| MAE | 0.0176 | 0.0130 |
| Similarità coseno | 0.945 | 0.954 |
| Tempo di inferenza | 26.35 ms | 47.61 ms |
| Cicli di CPU | 4.742.438 | 8.569.429 |
| Cicli per MACC | 3.05 | 3.09 |

Tabella 3.5: *Risultati della validazione on target sul test set (4.888 campioni)*

scheda (X-cross) dà il 100% di coerenza, quindi il codice eseguito sul microcontrollore riproduce il modello quantizzato.

Nella rete a due blocchi, le due convoluzioni pointwise occupano circa il 53% del tempo totale (5.07 ms e 8.92 ms), la prima convoluzione standard il 17% (4.56 ms) e le due depthwise il 13.5% ciascuna (3.57 ms), mentre pooling, dense e softmax pesano meno del 2% in tutto (Figura 3.14).

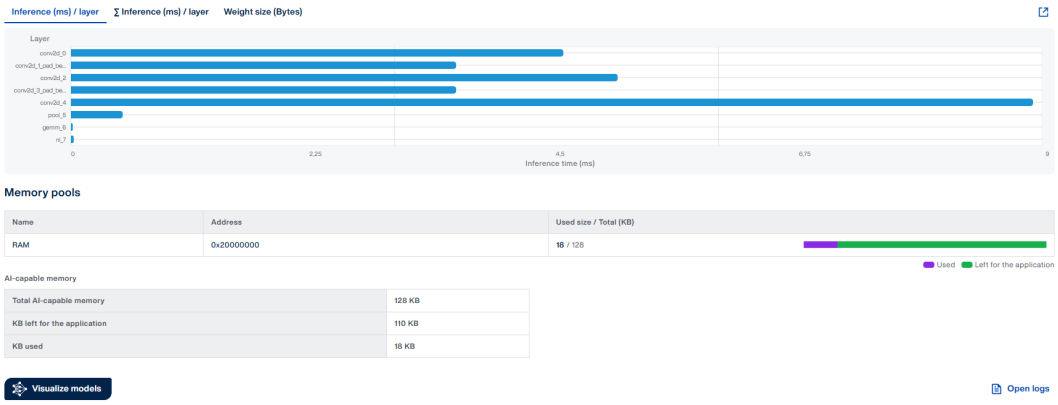



Figura 3.14: *Tempo di inferenza per layer e occupazione della RAM della rete a 2 blocchi, misurati on target*

## 3.10 Fase 10: Confronto tra rete a 2 blocchi e rete a 4 blocchi

La rete a quattro blocchi guadagna l'1.57% (93.94% contro 92.37%), ma costa circa il 74% di pesi in più (24 KB contro 14 KB), circa il 78% di MACC in più e circa l'81% di tempo di inferenza in più (47.61 contro 26.35 ms). La RAM non cambia. I valori di accuratezza sono coerenti con quelli ottenuti in fase di addestramento.

La Figura 3.15 mostra da dove viene il tempo in più: i due blocchi aggiunti ripetono gli stessi tempi per layer di quelli della rete a due blocchi, e quasi tutto l'aumento da 26.35 a 47.61 ms è dovuto alle loro convoluzioni depthwise e pointwise.

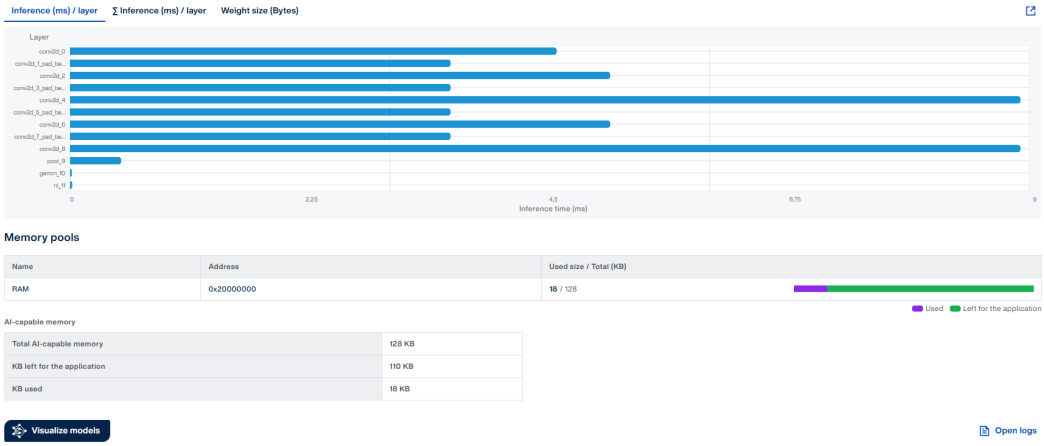



Figura 3.15: *Tempo di inferenza per layer e occupazione della RAM della rete a 4 blocchi, misurati on target*

Entrambe le reti hanno tempi di inferenza molto inferiori alla durata di 1 s del segmento audio su cui lavorano. Per questo motivo la rete a due blocchi è stata considerata il miglior compromesso tra occupazione di memoria e prestazioni per il dispositivo.

# Conclusioni

Il progetto aveva l'obiettivo di portare un sistema di keyword spotting a 12 classi su un microcontrollore STM32F446, con 512 KB di flash e 128 KB di RAM. Partendo dal Google Speech Commands abbiamo estratto le feature MFCC, addestrato una DS-CNN e convertito il modello in TFLite con quantizzazione int8, che riduce i pesi di circa il 73% rispetto al modello float. Il codice C generato da STM32Cube AI Studio è stato poi validato sia sul PC sia sulla scheda.

Entrambe le versioni della rete rientrano con ampio margine nelle risorse disponibili. La rete a due blocchi occupa 14 KB in flash e $\sim$18 KB in RAM, raggiunge il 92.37% di accuratezza sul test set e completa un'inferenza in 26.35 ms. La rete a quattro blocchi arriva al 93.94%, ma richiede 24 KB di pesi e 47.61 ms per inferenza, con la stessa RAM. In tutti e due i casi il codice eseguito sul microcontrollore riproduce il modello quantizzato di riferimento, con il 100% di X-cross correlation. Abbiamo scelto la rete a due blocchi perché, a fronte di un 1.57% di accuratezza in meno, usa circa il 43% di pesi in meno e impiega circa il 45% di tempo in meno per ogni inferenza. Con 26 ms su una finestra audio di 1 s, al microcontrollore resta comunque molto tempo libero per le altre elaborazioni (ad esempio, per il calcolo dei coefficienti MFCC della finestra audio successiva), mentre se in un'applicazione contasse di più l'accuratezza, anche la rete a quattro blocchi resterebbe utilizzabile.

Per il problema della porta seriale descritto nella Fase 9, la validazione on target è stata completata sulla Nucleo-F446RE e non sul BlueCoin: il microcontrollore e la frequenza sono gli stessi, quindi tempi e memoria sono rappresentativi. Inoltre la rete è stata validata su feature MFCC calcolate in anticipo sul PC, non su audio acquisito dalla scheda. Gli sviluppi futuri sono quindi la validazione sul BlueCoin, usando un canale di comunicazione collegato al suo microcontrollore, e l'acquisizione dell'audio dai microfoni della scheda con il calcolo delle MFCC direttamente sul dispositivo.

# Bibliografia


[1] Soumen Garai and Suman Samui. Advances in small-footprint keyword spotting: A comprehensive review of efficient models and algorithms. *arXiv preprint arXiv:2506.11169*, 2025.

[2] Yundong Zhang, Naveen Suda, Liangzhen Lai, and Vikas Chandra. Hello edge: Keyword spotting on microcontrollers. *arXiv preprint arXiv:1711.07128*, 2017.

[3] Pete Warden. Speech commands: A dataset for limited-vocabulary speech recognition. *arXiv preprint arXiv:1804.03209*, 2018.

[4] Google speech commands. `https://www.kaggle.com/datasets/sylkaladin/speech-commands-v2`. Consultato a luglio 2026.

[5] Mel-frequency cepstral coefficients (mfcc) for speech recognition. `https://www.geeksforgeeks.org/nlp/mel-frequency-cepstral-coefficients-mfcc-for-speech-recognition/`. Consultato a luglio 2026.

[6] Steven Davis and Paul Mermelstein. Comparison of parametric representations for monosyllabic word recognition in continuously spoken sentences. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 28(4):357–366, 1980.